Weakly-supervised Depth Estimation and Image Deblurring via Dual-Pixel Sensors

Liyuan Pan, Richard Hartley, Liu Liu, Zhiwei Xu, Shah Chowdhury, Yan Yang, Hongguang Zhang, Hongdong Li, and Miaomiao Liu

Abstract—Dual-pixel (DP) imaging sensors are getting more popularly adopted by modern cameras. A DP camera captures a pair of images in a single snapshot by splitting each pixel in half. Several previous studies show how to recover depth information by treating the DP pair as an approximate stereo pair. However, dual-pixel disparity occurs only in image regions with defocus blur which is unlike classic stereo disparity. Heavy defocus blur in DP pairs affects the performance of depth estimation approaches based on matching. Therefore, we treat the blur removal and the depth estimation as a joint problem. We investigate the formation of the DP pair, which links the blur and depth information, rather than blindly removing the blur effect. We propose a mathematical DP model that can improve depth estimation by the blur. This exploration motivated us to propose our previous work, an end-to-end **DDDNet** (DP-based Depth and Deblur Network), which jointly estimates depth and restores the image in a supervised fashion. However, collecting the ground-truth (GT) depth map for the DP pair is challenging and limits the depth estimation potential of the DP sensor. Therefore, we propose an extension of the DDDNet, called **WDDNet** (Weakly-supervised Depth and Deblur Network), which includes an efficient reblur solver that does not require GT depth maps for training. To achieve this, we convert all-in-focus images into supervisory signals for unsupervised depth estimation in our WDDNet. We jointly estimate an all-in-focus image and a disparity map, then use a *Reblur* and *Fstack* module to regularize the disparity estimation and image restoration. We conducted extensive experiments on synthetic and real data to demonstrate the competitive performance of our method when compared to state-of-the-art (SOTA) supervised approaches.

Index Terms—Dual-pixel Sensor, Weakly-supervised, Depth Estimation, Deblur and Reblur.

1 Introduction

Dual-pixel (DP) sensors are becoming more widely used by modern DSLR (digital single-lens reflex) and smartphone cameras. It captures two images of the scene in a single snapshot, using a unique design that splits each pixel in half with two photodiodes, resulting in a DP pair. For image regions outside the depth of field (DoF), the left and right views of a DP pair will show a disparity (Fig. 1, first column). Though the DP sensor is designed for auto-focus [1], [2], [3], [4], it is used in applications such as, depth estimation [5], [6], [7], [8], [9], defocus deblurring [10], [11], [12], reflection removal [13], and shallow Depth-of-Field (DoF) images synthesis [14]. In this paper, we theoretically model the imaging process of the DP sensor and demonstrate its effectiveness for simultaneous depth estimation and image deblurring.

A DP camera simultaneously captures two images, one formed from light rays passing through the right half of the aperture, and one from those passing through the left half of the aperture. Specifically, a set of points in the world that are exactly in focus forms a plane parallel to the lens (perpendicular to the axis of the camera). Points lying on this plane are both in focus, and imaged with zero-disparity in the two images. Points that lie further away from the plane are imaged with a disparity in the two images. In auto-focus, these disparities are used to move the sensor, or the lens, to focus on any part of the scene.

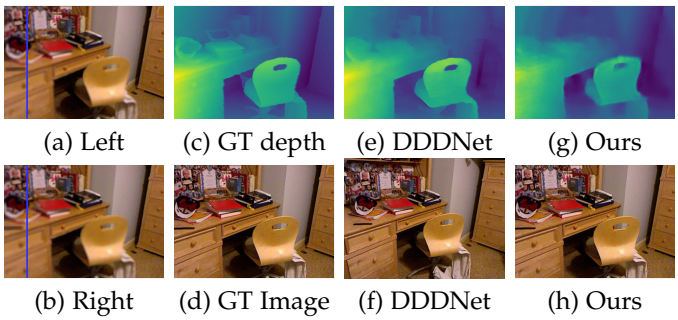


Fig. 1. *Our depth estimation and image restoration results. (a-b) The input DP pair. A blue line in each view of the DP image is drawn to indicate the small shift between the left and right sub-aperture views. (c-d) The GT depth map and all-in-focus image. (e-f) Depth estimation and image deblurring results of our supervised DDDNet [15]. (g-h) Depth estimation and image deblurring results of our weakly-supervised WDDNet. (Best viewed in colour on the screen.)*

Due to the displacement of the left and right half-apertures, the two images approximately form a stereo pair [8], [10]. Moreover, the small baseline between the two half-apertures makes occlusion effects minimal, which benefits disparity estimation. By detecting and modeling the disparity, it is possible to compute scene depth, as in standard stereo imaging [5], [7]. However, despite the resemblance to

- *Liyuan Pan is with the School of Computer Science and Technology, Beijing Institute of Technology, Beijing, China.*
  Richard Hartley, Zhiwei Xu, Hongdong Li, and Miaomiao Liu are with the School of Computing, Australian National University, Canberra, Australia.
  Liu Liu is with KooMap Dept., Huawei, Beijing, China.
  Shah Chowdhury is with Rajshahi University of Engineering & Technology, Bangladesh.
  Yan Yang is with the Biological Data Science Institute, Australian National University.
  Hongguang Zhang is with Systems Engineering Institute, AMS, China.
  E-mail: liyuan.pan@bit.edu.cn.

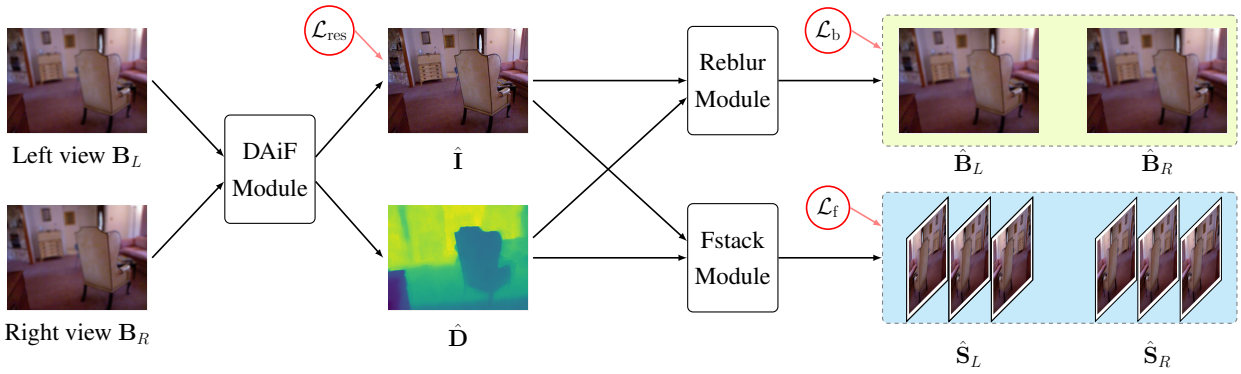


Fig. 2. *The architecture of our network. Given an input DP pair $\mathbf{B}_{\{L,R\}}$, the DAiF module estimates a disparity map $\hat{\mathbf{D}}$ and an all-in-focus sharp image $\hat{\mathbf{I}}$. With $\hat{\mathbf{D}}$ and $\hat{\mathbf{I}}$, we recover the input DP pair with the Reblur module, generating $\hat{\mathbf{B}}_{\{L,R\}}$. The same $\hat{\mathbf{D}}$ and $\hat{\mathbf{I}}$ are also used by the Fstack module, generating a focal stack $\hat{\mathbf{S}}_{\{L,R\}}$. During our unspervised training process, we use: 1) GT sharp image $\mathbf{I}$ to supervise $\hat{\mathbf{I}}$, yielding loss L_{res}; 2) Input $\mathbf{B}_{\{L,R\}}$ to supervise $\hat{\mathbf{B}}_{\{L,R\}}$ (L_b); and 3) 'GT' focal stack to supervise $\hat{\mathbf{S}}_{\{L,R\}}$ (L_f). Our network is trained end-to-end, without relying on GT disparity map.*

classic stereo, the nature of the disparity in DP sensors still has key differences from stereo disparity [6], [15], [16].

As discussed above, points other than those lying on the in-focus plane will also be significantly out of focus, which raises the possibility of using depth-from-defocus techniques to estimate the depth of 3D points in the scene [6], [17]. This out-of-focus effect is more significant than with ordinary stereo pairs, making stereo matching potentially more challenging for DP cameras. Therefore, computing depth from a DP camera can be seen as an **out-of-focus stereo** estimation problem. This paper aims to combine depth-from-disparity and depth-from-defocus approaches in a single network, demonstrating the advantage of modeling them simultaneously.

To achieve this, we proposed a DP-based Depth and Deblur Network, **DDDNet**, in our previous work [15]. Our DDDNet jointly recovers an all-in-focus image and estimates the depth of a scene by relating the desired all-in-focus image, the captured DP pair, and depth. Please refer to Fig. 1 for more details.

Although several learning-based methods, including ours, [5], [6], [7], tackle the depth estimation and deblurring problem in a data-driven manner and achieve promising results, collecting large and high-quality training data is challenging. To the best of our knowledge, only pseudo ground truth (GT) depth maps are available for existing DP datasets. For example, [5] uses structure from motion, and [6] estimates depth maps from the focal stacks. The work [15] and [18] develop simulators to synthesize DP data by using 'all-in-focus image + depth'. However, collecting high-quality GT depth is costly and time-consuming even with depth sensors, *e.g.*, LiDAR and Kinect. Moreover, methods that rely on synthetic data need to bridge the domain gap between synthetic and real DP images.

Therefore, this paper develops a DP-based weakly-supervised depth estimation and deblurring network, **WDDNet**, that does not rely on GT depth for training. Moreover, our model can mitigate domain gaps between synthesized and real images by training with real-world data without GT depth. Our key idea is to use the all-in-focus image as an intermediate step, or supervisory signal, to assist depth estimation from a DP pair. Compared to capturing high-quality GT depth maps, acquiring all-in-focus images is much easier, as all-in-focus images can be captured with a small aperture and long exposure [10], [15]. Our network (Fig. 2) consists of three modules:

1) A DAiF module to restore an all-in-focus image $\hat{\mathbf{I}}$ and an inverse depth map $\hat{\mathbf{D}}$. Given an input DP pair $\mathbf{B}_{\{L,R\}}$, the DAiF module estimates $\hat{\mathbf{I}}$ and $\hat{\mathbf{D}}$;
2) A Reblur module to recover the input DP pair. Given $\hat{\mathbf{I}}$ and $\hat{\mathbf{D}}$, the Reblur module estimates $\hat{\mathbf{B}}_{\{L,R\}}$;
3) A Fstack module to generate an image set with different focus positions. Given $\hat{\mathbf{I}}$ and $\hat{\mathbf{D}}$, the Fstack module synthesizes a DP focal stack $\hat{\mathbf{S}}_{\{L,R\}}$.

To enable weakly-supervised learning, we supervise the outputs of the above three modules: i) The GT all-in-focus image $\mathbf{I}$ is used to ensure the correctness of $\hat{\mathbf{I}}$, to solve the ill-posed problem of simultaneously estimating $\hat{\mathbf{I}}$ and $\hat{\mathbf{D}}$ from $\mathbf{B}_{\{L,R\}}$; ii) The input DP pair $\mathbf{B}_{\{L,R\}}$ is used to regularize $\hat{\mathbf{B}}_{\{L,R\}}$. Since our Reblur module mathematically models the DP imaging process, only with correct depth estimation $\hat{\mathbf{D}}$ can our Reblur module generate a correct DP pair; iii) $\mathbf{I}$ is further used with estimated $\hat{\mathbf{D}}$, generating a 'GT' focal stack, to supervise $\hat{\mathbf{S}}_{\{L,R\}}$. Supervising the focal stack $\hat{\mathbf{S}}_{\{L,R\}}$ helps to further regularize our network at each focal plane and improves the quality of the restored all-in-focus image $\hat{\mathbf{I}}$. In order to model the DP imaging process, we develop a straightforward simulator that seamlessly integrates into the inner loop of a neural network training process. Given that the integral process is acquired for synthesizing the DP image within our simulator, we employ the summed-area table technique [19], [20] for efficient handling of integrals.

Our contributions are summarised as follows:

1) We design an end-to-end DP-based weakly-supervised depth estimation and deblurring network (WDDNet) based on the DP sensor;
2) We propose a Reblur and Fstack module to turn all-in-focus images into supervisory signals for unsupervised depth estimation. We explicitly derive the backward gradients to enable end-to-end training. With pixel extrapolation in a continuous space and pixel-level CUDA parallelism, the two modules are implemented with high efficiency;
3) Experimentally, our WDDNet is evaluated extensively on both synthetic and real data. Our weakly-supervised

model achieves competitive performance compared to the state-of-the-art (SOTA) supervised depth estimation methods. Our restored image quality significantly outperforms the SOTA methods.

2 Related Work

For space reasons, we briefly review stereo and monocular depth estimation. Then, we introduce depth from defocus, deblurring and DP sensors-based methods.

Stereo. Depth estimation is a fundamental problem in computer vision. Similar to humans, a machine can recover dense depth maps from a stereo camera [21], [22], [23], [24], [25]. To summarize, a stereo depth estimation framework [26], [27], [28] often contains: matching cost computation, cost aggregation, and disparity optimization. Recently, several speed-up methods [29], [30] are available.

Monocular. Monocular depth estimation is significantly more under-constrained, while the advent of deep learning saw rapid advancements. Supervised monocular depth estimation methods typically rely on large training datasets [31], [32]. Self-supervised methods [33], [34], [35], [36] often use estimated depths and camera poses to generate synthesized images, serving as supervision signals. However, producing high-quality depth from a single image remains difficult, due to the inherent ambiguities of monocular depth estimation.

Depth from defocus and deblurring. Depth from defocus has the same geometric constraints as disparity but different physiological constraints [37]. The estimated depth, also dubbed as defocus map, is commonly used to guide the deblurring [38], [39], [40], [41], [42], [43], [44]. DMENet [45] proposes the first end-to-end CNN architecture to estimate a spatially varying defocus map and use it for deblurring. Son *et al.* [46] restore sharp images from defocus blurred images by estimating spatial pseudo inverse kernels.

Dual-pixel. In recent studies, researchers have investigated depth estimation from DP data. The two-view DP images form an image pair with a tiny baseline and also suffer from defocus blur effects. As a pioneering effort, Wadhwa *et al.* [14] first applied classical stereo matching methods to DP views to compute depth by utilizing a small search size in block matching. They assume that *'DP sensors effectively create a crude, two-view light field with a baseline the size of the mobile camera's aperture'*. Similar descriptions are made by Garg *et al.* [5] and Zhang *et al.* [7], that they treat the DP image pair as *'a tiny baseline binocular stereo pair'* or *'a kind of stereo pair'*. Garg *et al.* [5] present the first learning models for affine invariant depth estimation to tackle affine ambiguity and Zhang *et al.* [7] present a Du^{2}Net that uses a wide baseline stereo camera to compensate for the DP camera. As illustrated in [8], a DP sensor can be treated as *'a two-sample light field camera, or a stereo system with a tiny baseline'*. Therefore, they [8] split DP image to layers and jointly estimate the all-in-focus image and defocus map with pre-calibrate blur kernels. Abuolaim *et al.* [10], [11], [18] claim *'DP sensors consist of two photodiodes at each pixel location effectively providing the functionality of a simple two-sample light-field camera'*. To model the defocus blur in a DP pair for depth estimation, Abuolaim *et al.* [10] first propose a DPDNet that removes the blur effect blindly. Punnappurath *et al.* [6] use a point spread function (PSF) to model the defocus blur and formulate the DP disparity by the PSF. However, the symmetry assumption of the PSF (which only holds for constant depth region) limits its application in a real-world scenario. Moreover, the method is time-consuming and has three steps (not end-to-end). Lee *et al.* [47] implicitly estimate the disparity of the left/right views using a spatial transformer, yet only in feature space, as well as Yang *et al.* [16]. Our previous work [15] studies the imaging process of a DP pair and gives a mathematical model jointly handling the intrinsic problem of *depth from defocus blur* for a DP pair while in a supervised manner. Liang *et al.* [17] use fixed network branches for layer-wise defocus map estimation and deblurring, where the limited network branch number restricts the potential of its accuracy.

DP data and Simulator. Deep learning-based methods significantly improve the quality of computer vision tasks with a large amount of training data. However, collecting real-world data is often costly and time-consuming. Only Canon and Google provide DP data to customers, though most DSLR and smartphone cameras have the DP sensor. Several researchers [5], [7], [14] use 'Google Pixel' to collect data. However, smartphone cameras use a fixed and narrow aperture that cannot be adjusted. Canon is used in [6], [10], [13] with different aperture sizes, but it is expensive. both the two-sensor is hard to get the associated GT information, such as depth. Therefore, several works target at synthesizing DP data to provide sufficient data for training. Pan *et al.* [15] present the first DP simulator that synthesizes DP images with the GT depth map and RGB image from any RGB-D data. Abuolaim *et al.* [18] propose a learning-based DP simulator, and they define a parametric PSF model based on the Butterworth filter to generate DP views.

3 Dual-Pixel Image Formation

In this section, we first introduce the DP model in Section 3.1 and 3.2. Then, we discuss the formation of the two DP images and the defocus blur in Section 3.3. In Section 3.4, we present our DP model and explore how to synthesize DP images from the RGB-D image. Based on the DP model, we build a DP simulator in Section 3.5.

3.1 A new look at Dual pixel

A DP camera can be modelled as an ordinary camera with a lens satisfying the thin-lens model, in which two (or more) images $\mathbf{I}_L$ and $\mathbf{I}_R$ are simultaneously captured. Here, the subscripts $_L$ and $_R$ denote the left and right views separately. In the model, the focal plane of the camera can be thought of as consisting of two identically placed focal planes, one of which captures light rays coming from the left side of the lens, and the other captures light from the right side of the lens. Therefore, the DP camera cannot be considered as a pinhole camera; rather, the lens must have a finite size.

The two images, $\mathbf{I}_L$ and $\mathbf{I}_R$ can be thought of as images taken with two slightly-different (but coplanar) lenses, corresponding to regions in the aperture of a thin-lens. To be general, let it be assumed that the light for image $\mathbf{I}_L$

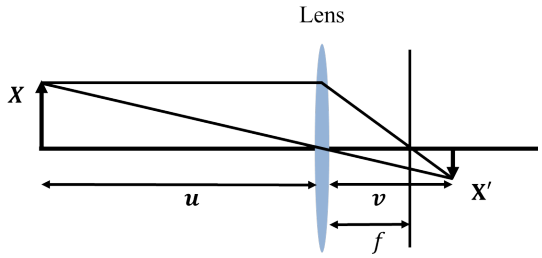


Fig. 3. *A point* $\mathbf{X}$ *at distance* u *on one side of a lens with focal length* f *is focused at a point* $\mathbf{X}'$ *at a distance* v *on the other side of the lens, where* $1/u + 1/v = 1/f$*. Thus every ray from* $\mathbf{X}$ *that meet the lens is refracted and passed through* $\mathbf{X}'$*. We set a coordinate frame such that the lens lies on the plane* $\mathrm{X} = 0$*, and the centre of the lens is at the origin. Let the point* $\mathbf{X} = (\mathrm{X}, \mathrm{Y}, \mathrm{Z})$ *lie to the left of the lens (that is with* $\mathrm{X} < 0$*), and the camera lies on the right of the lens.*

passed through a region $\mathbf{A}_L$ of the lens, and that for image $\mathbf{I}_R$ comes from a region $\mathbf{A}_R$.

Our model of the image formation process for dual-pixel cameras is essentially in agreement with the ones observed in previous research works [5], [7], [8], [10], [11], [14], [18]. Specifically, each pixel location on the DP sensor has two photodiodes that are separated and designed to record the signal independently. A light ray from a scene point within the camera's depth of field (DoF) would hit the two photodiodes with zero phase difference, forming a single-pixel circle of confusion (CoC) and zero disparity. Light rays outside the DoF would have a difference in phase correlated with the CoC size. Notably, the two optical paths are slightly different – no matter how small the difference is (otherwise, the two pixels would capture the same light ray). Therefore, existing works approximately treat the DP pair as a kind of stereo pair for simplification. Then, pointed out by Punnappurath *et al.* [6], *'unlike in stereo, DP disparity is the difference in the point spread functions (PSF) that produces disparity between DP views and not an explicit shift in image content'*. The emphasis of DP disparity aims to measure the DP disparity by making the symmetry assumption of the DP PSF (kernel). However, the mathematical formulation for synthesizing DP pairs is still under-explored.

In our work, we first approximate the DP pair as an out-of-focus stereo pair to measure the DP disparity. This helps us to put our formulation into equations with the above-simplified setup for the DP sensor. Therefore, we can formulate a simple simulator which can fit into the inner loop of a neural network training process and allow us to get better results. Specifically, we can quantitatively synthesize the images to assist in computing the DP disparity without explicitly calculating DP kernels. In the following, we start with illustrating the DP model theory with two optical centres, which is a simplified setup for DP sensors. Then, for a real-world DP lens with left and right apertures, we use two regions to model it. Finally, integrating all ray intensities from regions gives a blurry DP pair.

3.2 Projective geometry of a thin lens

As a theoretical preliminary, it will be shown that a thin lens creates a 3D image of the 3D world, which is a 3D projective transformation of the world. This 3D projective transform can be represented by a 4×4 homogeneous matrix.

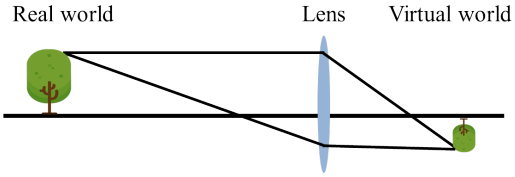


Fig. 4. *Light emanating from the real world* W *is focused to form a virtual image* W′ *of the world inside the camera, beyond the focal length of the lens.*

Note that this is not the same as the 3D to 2D projective transform from world to image, represented by a 3×4 matrix, familiar in geometric computer vision.

Thin lens equation. The equation of a thin lens is familiar to elementary optics. Consider diagram Fig. 3. The basic lens equation is given, as explained in Fig. 3 by

$$\frac{1}{f} = \frac{1}{u} + \frac{1}{v} \,. \tag{1}$$

A ray through the centre of the lens does not deviate, and continues in a straight line (using a thin lens assumption). This allows us to determine where rays from the point $\mathbf{X}^{'}$ will focus.

Let us choose a coordinate frame such that the lens lies on the plane $\mathrm{X} = 0$, and the centre of the lens is at the origin. In addition, let the "world" lie to the left of the lens (that is with points $\mathrm{X} < 0$), and the camera lies on the right of the lens.

Consider a point $\mathbf{X} = (\mathrm{X}, \mathrm{Y}, \mathrm{Z})$[1], which is focused at point $\mathbf{X}^{'} = (\mathrm{X}^{'}, \mathrm{Y}^{'}, \mathrm{Z}^{'})$. The thin lens equation give, setting $u = -\mathrm{X}$, and $v = \mathrm{X}^{'}$:

$$-\frac{1}{\mathrm{X}} + \frac{1}{\mathrm{X}'} = \frac{1}{f} \,. \tag{2}$$

The minus sign is present here, because in the lens equation, the distances u and v are measured on opposite sides (different directions) from the lens. From this, we get $\mathrm{X}' = f\mathrm{X}/(f + \mathrm{X})$.

Because of the condition that a ray through the origin is undeviated, we see that

$$\begin{aligned} \frac{\mathrm{Y}}{\mathrm{X}} &= \frac{\mathrm{Y}^{'}}{\mathrm{X}'}, \\ \frac{\mathrm{Z}}{\mathrm{X}} &= \frac{\mathrm{Z}^{'}}{\mathrm{X}'} \,. \end{aligned} \tag{3}$$

From this, we get the equations

$$\begin{aligned} \mathrm{X}' &= f\mathrm{X}/(f + \mathrm{X}) \,, \\ \mathrm{Y}' &= f\mathrm{Y}/(f + \mathrm{X}) \,, \\ \mathrm{Z}' &= f\mathrm{Z}/(f + \mathrm{X}) \,. \end{aligned} \tag{4}$$

This can be written as a projective transform written in terms of a matrix acting on homogeneous coordinates as

$$\begin{pmatrix} \mathrm{X}' \\ \mathrm{Y}' \\ \mathrm{Z}' \\ \mathrm{W}' \end{pmatrix} = \begin{bmatrix} 1 & & & \\ & 1 & & \\ & & 1 & \\ 1/f & & & 1 \end{bmatrix} \begin{pmatrix} \mathrm{X} \\ \mathrm{Y} \\ \mathrm{Z} \\ 1 \end{pmatrix} \,. \tag{5}$$

The formation of the 3D virtual image (actually a real image in standard terminology), by the lens, is illustrated in Fig. 4.

A few observations are in order here.

1. All vectors are column vectors.

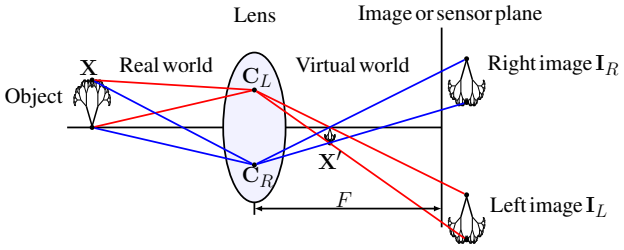


Fig. 5. *Formation of the two image $\mathbf{I}_L$ and $\mathbf{I}_R$ from two points $\mathbf{C}_L$ and $\mathbf{C}_R$ on the lens plane may be thought of as pin-hole projections (with centres $\mathbf{C}_L$ and $\mathbf{C}_R$ and focal-length F) of the virtual image formed by the lens.*

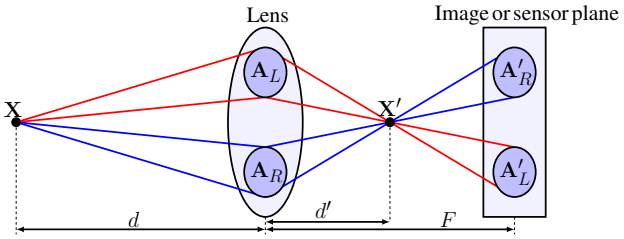


Fig. 6. *Rays from a point $\mathbf{X}$ in the scene at depth d pass through the aperture $\mathbf{A}_L$ and are focused at a point $\mathbf{X}'$ at depth d' in the virtual world W'. The set of all the rays passing through $\mathbf{A}_L$ form, after refraction, a double-sided cone, with vertex at $\mathbf{X}'$. This cone, meets the focal plane at depth F in a region that is geometrically similar to the shape of $\mathbf{A}_L$. This explanation is continued in Fig. 7.*

1) A world point $\mathbf{X} = (\mathrm{X}, \mathrm{Y}, \mathrm{Z})$ with $\mathrm{X} < -f$ maps to a real image point in the right half-plane.
2) A point with $\mathrm{X} = -f$ maps onto the plane $\mathrm{X}' = \infty$.
3) A world point with $-f < \mathrm{X} < 0$ maps to a virtual point X', in the left-half plane.

This shows that a thin lens makes a projective transformation of the world, creating a projectively distorted 3D image of the world. In particular, for points $\mathbf{X}$ with $\mathrm{X} < -f$, the camera creates a real (as opposed to virtual) image, in the right half-plane (that is, on the other side of the lens).

Despite the fact that this is a real image, not a virtual image, we shall call this the virtual world W' formed by the cameras.

3.3 Model of a dual-pixel camera

A dual-pixel camera can be considered as forming two images $\mathbf{I}_L$ and $\mathbf{I}_R$, formed using rays that pass through regions $\mathbf{A}_L$ and $\mathbf{A}_R$ in the lens plane. Let us consider the case where $\mathbf{A}_L$ and $\mathbf{A}_R$ are infinitesimally small, consisting of just two points $\mathbf{C}_L$ and $\mathbf{C}_R$. What do the two images $\mathbf{I}_L$ and $\mathbf{I}_R$ consist of in this case (see Fig. 5)?

Let W be the "world", consisting of points $\mathbf{X}$ with $\mathrm{X} < -f$, and let W' be the 3D image of the world, created by the lens, lying to the right of the lens. In particular, if $\mathbf{X}' = (\mathrm{X}', \mathrm{Y}', \mathrm{Z}') \in \mathrm{W}'$, then $\mathrm{X}' > f$.

Consider a plane (called a focal plane) lying inside the camera [48]. Assume that this is parallel to the lens, so it is a plane defined by the equation $\mathrm{x} = \mathrm{F}$. with $F > 0$. The symbol of F is chosen to imply that this is the distance of the focal plane from the lens, but it is not the same as the focal length of the lens, which is f. It will now be explained what the two images formed on the focal plane corresponding to rays passing through regions $\mathbf{A}_L$ and $\mathbf{A}_R$ are.

First, we consider the case where $\mathbf{A}_L$ and $\mathbf{A}_R$ are infinitesimally small, consisting of single points $\mathbf{C}_L$ and $\mathbf{C}_R$.

Observation. *An image $\mathbf{I}_L$ of the world* W *formed from rays passing through a point $\mathbf{C}_L$ in the focal plane is the same as a pin-hole camera image of the virtual world W' with projection centre $\mathbf{C}_L$ and the same focal plane. The same statement is true for the image $\mathbf{I}_R$ formed from rays passing through $\mathbf{C}_R$.*

Proof. Refer to Fig. 5. Consider a world point $\mathbf{X}$, mapped by the lens to a virtual world point $\mathbf{X}'$. This means that if $\mathbf{C}$ is any point on the lens, then the ray $\mathbf{XC}$ is refracted by the lens to the ray $\mathbf{CX}'$. In particular, this is true for the point $\mathbf{C}_L$ lying on the lens. Let the point $\mathbf{Y}$ be the point where the ray $\mathbf{C}_L\mathbf{X}'$ (extended if necessary) meets the focal plane. Then $\mathbf{Y}$ is the point where $\mathbf{X}$ is imaged in $\mathbf{I}_L$. This is because a ray traverses the path $\mathbf{XC}_L\mathbf{Y}$, passing through $\mathbf{X}'$ on the way. On the other hand, looked at in terms of the pinhole camera with centre $\mathbf{C}_L$, the ray from $\mathbf{C}_L$ to $\mathbf{X}'$ meets the focal plane at $\mathbf{Y}$. This means that $\mathbf{Y}$ is the image of the point $\mathbf{X}'$ in the pinhole image taken from $\mathbf{C}_L$. □

Therefore, if the two regions were single points $\mathbf{C}_L$ and $\mathbf{C}_R$, then the two images $\mathbf{I}_L$ and $\mathbf{I}_R$ are a stereo pair of images of the virtual world W'. Note that the two regions in a DP camera are not single points; therefore, the two DP images $\mathbf{I}_L$ and $\mathbf{I}_R$ cannot form an exact stereo pair. Although the two points $\mathbf{C}_L$ and $\mathbf{C}_R$ are necessarily very close together, the virtual world is also very small and very close to the camera lens, so there will be appreciable disparity.

Non-pinhole model. In the case where the two regions $\mathbf{A}_L$ and $\mathbf{A}_R$ are bigger than a pinhole, and in fact constitute half the lens itself, the images $\mathbf{I}_L$ and $\mathbf{I}_R$ formed will be made up as the superposition of images of the virtual world W' taken at all points $\mathbf{C}_L$ in region $\mathbf{A}_L$ and $\mathbf{C}_R$ in region $\mathbf{A}_R$. They will consequently be blurred. This is shown in Fig. 6.

Therefore, the problem of finding depth in the scene from images $\mathbf{I}_L$ and $\mathbf{I}_R$ is equivalent to doing stereo from blurred images. This will have its problems. The blur will be depth-dependent. Points in the virtual world that lie on the focal plane π will be in focus independent of the position of the point $\mathbf{C}_L$ and $\mathbf{C}_R$. Thus, points in the world corresponding to this placement of the focal plane will be both in focus and identically positioned in the two images $\mathbf{I}_L$ and $\mathbf{I}_R$. Points that lie off the focal plane will be blurred and, at the same time, displaced by a disparity. Determining depth from the pair of images is, therefore, a hybrid problem, consisting of stereo in the presence of substantial depth-dependent blur.[2]

3.4 Image synthesis from RGB-D image

Given an image with an associated depth map, it is possible to synthesize either of the pair of dual-pixel images. Consider the image $\mathbf{I}$, formed from all rays passing through a region $\mathbf{A}$ in the plane of the lens. (Note, $\mathbf{A}$ is either $\mathbf{A}_L$ or $\mathbf{A}_R$, and the corresponding image is either $\mathbf{I}_L$ or $\mathbf{I}_R$.)

2. The problem will be easier to solve if the position of the focal plane F is swept from back to front (the varying focus of the lens) mechanically during the capture of the images. In this case, the problem of depth determination will be a sort of the hybrid problem between depth-from-stereo and depth-from-focus.

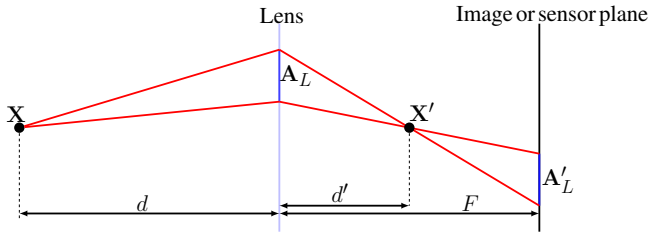


Fig. 7. *Viewed side-on, rays from a point* $\mathbf{X}$ *pass through the aperture* $\mathbf{A}_L$ *(similarly* $\mathbf{A}_R$*) and are focused at* $\mathbf{X}'$*. They meet the focal plane at depth* F *(which may be either side of* $\mathbf{X}'$*) covering a region* $\mathbf{A}'_L$ *geometrically similar to* $\mathbf{A}_L$*. The scale of* $\mathbf{A}'_L$ *is computed by similar triangles; the scale factor is given by* $(d' - F)/d'$*. This scale factor is positive if* F *lies in front of* $\mathbf{X}'$ *and negative if it lies behind; in this letter case,* $\mathbf{A}'_L$ *is inverted with respect to* $\mathbf{A}_L$*.*

Let $\mathbf{I}_W$ be an RGB-D image of the world, taken nominally from the viewpoint of the lens centre. Since the lens is normally very small with respect to the world, it will be assumed that any world point visible from the lens centre will be visible from any other point on the lens.

The RGB-D image gives us 3D coordinates of any point visible in the image. Let $\mathbf{X} = (\mathrm{X}, \mathrm{Y}, \mathrm{Z}) \in \mathrm{W}$ be such a point. The image of this point in the virtual world W' is given by

$$\mathbf{X}' = (\mathrm{X}', \mathrm{Y}', \mathrm{Z}') = \frac{f}{f + \mathrm{X}}(\mathrm{X}, \mathrm{Y}, \mathrm{Z}). \tag{6}$$

This point is then projected from each point $\mathbf{C} \in \mathbf{A}$ onto the focal plane $\mathrm{X} = F$. The projection rays, through $\mathbf{X}'$ from points in $\mathbf{A}$ form a cone with cross-section $\mathbf{A}$ and vertex $\mathbf{X}'$. This cone meets the focal plane in a further cross-section $\mathbf{A}'$. Since the lens and focal plane are parallel, $\mathbf{A}$ and $\mathbf{A}'$ will be similar regions, in the sense that there is a similarity transform relating $\mathbf{A}$ and $\mathbf{A}'$. Even more, this is simply a scaling and translation (where the scale may be positive or negative). In particular, a point $\mathbf{C} \in \mathbf{A}$ maps to a point $\mathbf{C}' = s\mathbf{C}+\mathbf{t}$. Here, $\mathbf{t}$ is a 2-dimensional offset and s is a scale. For a given point $\mathbf{X}$, the values of s and $\mathbf{t}$ are constant, not dependent on the particular point $\mathbf{C}$ chosen, but they vary according to the point $\mathbf{X}$ chosen.

Refer to Fig. 7. In particular, by similar triangles [49]

$$\mathbf{C}' = T(\mathbf{C}) = (1 - s)\mathbf{C} + s\mathbf{X}' \,, \tag{7}$$

where $s = F/d'$.

Image coordinates. We start with an RGB-D image, so each pixel in an image comes with an associated depth. For simplicity, we assume that all distances (including depth) are measured in pixel coordinates.

We make the following assumptions. A pixel with coordinates [3] (y, z) corresponds with a ray in space defined by $-d(1, y/f, z/f)$ for varying d. In particular, since each point in the image comes with a depth, we are assuming that the point imaged at point (y, z) has 3D coordinates $(-d, -dy/f, -dz/f)$. This corresponds to a pinhole camera with a focal plane given by $\mathrm{X} = f$ and a camera centre (pinhole) at the origin.

3. Considering our coordinate frame, it is convenient to use (y, z) for image coordinates, instead of the usual (x, y).

A point $\mathbf{X} = -d(1, y/f, z/f)$ at depth d will be mapped to a point $\mathbf{X}' = d'(1, y/f, z/f)$, where

$$\frac{1}{d'} = \frac{1}{f} - \frac{1}{d} \tag{8}$$

$$d' = \frac{fd}{d - f} \,. \tag{9}$$

The points $\mathbf{X}$ and $\mathbf{X}'$ are therefore given by

$$\mathbf{X} = \frac{d}{f}(f, y, z) \tag{10}$$

$$\mathbf{X}' = \frac{d}{d - f}(f, y, z) \,. \tag{11}$$

Let $\mathbf{C} = (0, \mathrm{Y}_0, \mathrm{Z}_0)$ be a point in $\mathbf{A}$ (lying on the lens plane $\mathrm{X} = 0$). The line $T(\mathbf{X}, \mathbf{C})$ from $\mathbf{C}$ through $\mathbf{X}' = d'(1, y/f, z/f)$ is expressed as $(1 - s)\mathbf{C} + s\mathbf{X}'$ for varying values of s. This line meets the plane $\mathrm{X} = F$ when $s = F/d'$. The coordinates of this point are therefore

$$T(\mathbf{X}, \mathbf{C}) = \frac{d' - F}{d'}\mathbf{C} + \frac{F}{d'}\mathbf{X}' \,. \tag{12}$$

and substituting for d' gives

$$T(\mathbf{X}, \mathbf{C}) = \left(1 - \frac{F}{f} + \frac{F}{d}\right)\mathbf{C} + \left(\frac{F}{f} - \frac{F}{d}\right)\mathbf{X}' \,. \tag{13}$$

Substituting for $\mathbf{X}'$ from (11) gives

$$T(d, y, z, \mathbf{C}) = \left(1 - \frac{F}{f} + \frac{F}{d}\right)\mathbf{C} + F\left(1, \frac{y}{f}, \frac{z}{f}\right) \,, \tag{14}$$

which expresses the projected point in terms of the parameters (F and f), the position of the aperture point $\mathbf{C}$ and the coordinates (y, z) and depth d of the pixel.

We are only interested in the (Y, Z)-coordinates, in which case we have

$$T(d, y, z, \mathbf{C}) = \frac{d' - F}{d'}(\mathrm{Y}_0, \mathrm{Z}_0) + F\left(\frac{y}{f}, \frac{z}{f}\right) \,. \tag{15}$$

Here, the transformation function $T(\mathbf{X}, \mathbf{C})$ for the left and right image has been defined with points $\mathbf{C}_L$ and $\mathbf{C}_R$ in $\mathbf{A}_L$ and $\mathbf{A}_R$ respectively.

Consider the image of a point as projected through two aperture points $\mathbf{C}_L$ and $\mathbf{C}_R$ separated by a *baseline distance* b. From (14), the distance between two projected points is

$$\|T(\mathbf{X}, \mathbf{C}_L) - T(\mathbf{X}, \mathbf{C}_R))\| = \left(1 - \frac{F}{f} + \frac{F}{d}\right)\|\mathbf{C}_L - \mathbf{C}_R\| \,, \tag{16}$$

which can be written as

$$D = \left(1 - \frac{F}{f} + \frac{F}{d}\right) b \,, \tag{17}$$

where D is the disparity. Our formulation puts into equations with above simplified setup for the DP sensor. Thus, we can actually quantitatively synthesize the images.

3.5 DP simulator

In this section, our goal is to formulate the simplest simulator which can fit into the inner loop of a neural network training process and allow us to get better results.

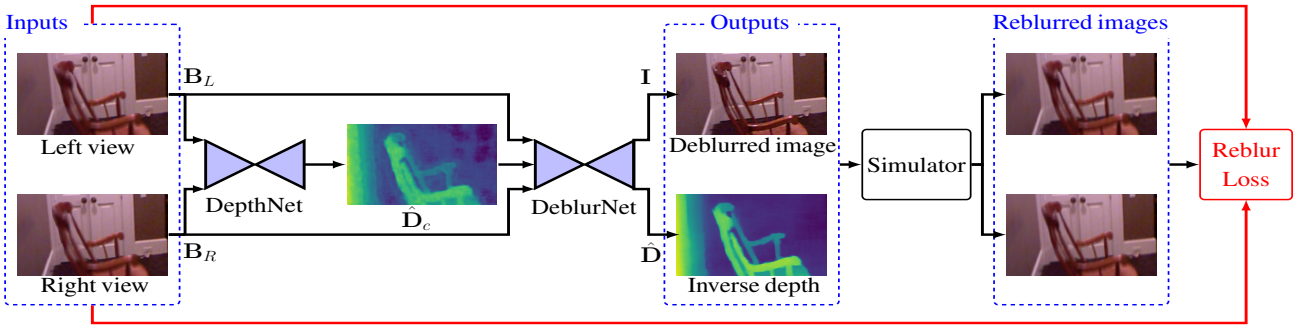


Fig. 8. *The architecture of the DDDNet. With the input DP image pair, the DepthNet first estimates a coarse inverse depth map $\hat{\mathbf{D}}_c$. Then, we combine the coarse and blurred inverse depth map with $\mathbf{B}_{\{L,R\}}$, and feed them to the DeblurNet. The deblurred image and estimated inverse depth map are fed to our DP simulator to synthesize a DP image pair. The synthesized DP images are compared to the inputs using our reblur loss (Eq. (23)), to regularise the DDDNet in training. Note that the ground truth inverse depth maps and sharp images are used as supervision signals in training.*

3.5.1 Differential images

As mentioned, the creation of a synthetic blurred image with a depth-dependent blur kernel can be accelerated using the idea of a differential image. Instead of creating the blurred image directly, one computes its derivative, which is then integrated (summed across the rows, and down the columns) to create the final blurred image. This is also closely related to 'summed-area tables' in graphics [50]. Summed-area tables enable the rapid calculation of the sum of the pixel values in an arbitrarily sized, axis-aligned rectangle at a fixed computational cost [19], [51]. A few more details are given here.

As has been shown, the blur kernel corresponding to a point $\mathbf{X}$ in the world is the intersection of a double-sided cone with vertex $\mathbf{X}'$ and cross-section the aperture $\mathbf{A}$ in the lens plane, intersected with the focal plane. As such, it is geometrically similar to $\mathbf{A}$, and will be denoted by $\mathbf{R}$, and called the blur-mask. The mask $\mathbf{R}$ is a region in the focal plane, and it is assumed that the intensity is evenly spread across $\mathbf{R}$.

Given blur mask $\mathbf{R}$, (a region in the focal plane) its *differential mask* is simply the output of a 2×2 filter, namely

$$\mathcal{I}(\mathbf{R}) = \mathbf{R} * \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix}$$

where $*$ represents convolution. Since this is a separable filter, it can be implemented as two 1-dimensional filters on the rows and columns of $\mathbf{R}$. Given $\mathcal{I}(\mathbf{R})$ one can retrieve $\mathbf{R}$ by integrating $\mathcal{I}(\mathbf{R})$ along the rows and columns.

For a binary mask $\mathbf{R}$ (with values 1 inside of $\mathbf{R}$ and 0 outside), its differential mask $\mathcal{I}(\mathbf{R})$ has value 0 inside or outside $\mathbf{R}$, and non-zero values **only on the boundary** of $\mathbf{R}$. Thus, the number of non-zero entries of $\mathcal{I}(\mathbf{R})$ is usually very much smaller than the number of non-zero entries of $\mathbf{R}$ itself. For instance, the differential mask of a rectangle contains only 4 non-zero entries, namely values 1 at the top-right and bottom left, and -1 at the other two corners.

In the case of more complicated apertures A (such as the half-circle represented by the half-aperture of a lens), one can approximate the shape of $\mathbf{R}$ by a small number of rectangles. This will cause the number of non-zero entries in the differential mask to increase, but it will still be significantly smaller than the number of zeros in the full mask $\mathbf{R}$. The shape $\mathbf{R}$ can be approximated to a desired degree of accuracy by using sufficiently many rectangles, or else the full differential mask of $\mathbf{R}$ can be used for full accuracy.

3.5.2 Synthesizing the DP image pair

Given a sharp image $\mathbf{I} \in \mathbb{R}^{H \times W}$ and its associated depth map $\mathbf{d} \in \mathbb{R}^{H \times W}$, we can simulate a DP image pair $\mathbf{B}_{\{L,R\}}$ as follows. Here, H and W are the image height and width, and it is also assumed that focal length f and focal-plane-sensor distance F are known (measured in units of pixel size).

For each pixel (y, z) in $\mathbf{I}$, as shown in Fig. 7 the intensity of a pixel is spread over regions $\mathbf{R}_L$ and $\mathbf{R}_R$, in the left and right view of the DP pair. Each region contains a set of points $\mathbf{p}$, containing $|\mathbf{R}|$ pixels, and the intensity $\mathbf{I}(y, z)$ of the pixel (y, z) is spread out evenly over this set of pixels. Now running over all pixels $\mathbf{I}(y, z)$, and summing, a pair of dual-pixel images is created. This is essentially blurring with a depth-dependent blur kernel.

This operation is expensive computationally, since it requires looping over each pixel (y, z) in the image, as well as each pixel in the region $\mathbf{R}$, thus 4 levels of looping.

We utilize the concept of 'integral image' to speed up the computation, so that it has complexity independent of the size of the region $\mathbf{R}$, thus of order $\mathcal{O}(n)$ where n is the number of pixels. In this description, we assume that the left and right halves of the aperture are approximated by rectangles, though with some amount of extra computation, more general shapes may be accommodated.

Given a pixel (y, z) of the RGB-D image at a given depth, its corresponding light rays will pass through every point of $\mathbf{A}_L$. To compute the blur-region $\mathbf{R}_L$, one needs only to compute the destinations of the rays passing through the four corners of $\mathbf{A}_L$, which will be denoted by $\mathbf{p}_{tl} = (y_{tl}, z_{tl})$, $\mathbf{p}_{tr} = (y_{tr}, z_{tr})$, $\mathbf{p}_{bl} = (y_{bl}, z_{bl})$ and $\mathbf{p}_{br} = (y_{br}, z_{br})$ in the left image $\mathbf{B}_L$. The positions of these four points are given by Eq. (15). Here, the subscript $_{tl}$, $_{tr}$, $_{bl}$ and $_{br}$ denotes top-left, top-right, bottom-left and bottom-right, respectively.

We create a *differential mask* $\mathcal{I}_L \in \mathbb{R}^{H \times W}$ for the region $\mathbf{R}_L$ defined by

$$\begin{aligned} \mathcal{I}_L(\mathbf{p}_{tl}) &= \mathbf{I}(y, z)/|\mathbf{R}_L|, \quad & \mathcal{I}_L(\mathbf{p}_{tr}) &= -\mathbf{I}(y, z)/|\mathbf{R}_L|, \\ \mathcal{I}_L(\mathbf{p}_{bl}) &= -\mathbf{I}(y, z)/|\mathbf{R}_L|, \quad & \mathcal{I}_L(\mathbf{p}_{br}) &= \mathbf{I}(y, z)/|\mathbf{R}_L| \,. \end{aligned} \tag{18}$$

These values are summed for the regions $\mathbf{R}_L$ corresponding to all points (y, z) in the image, to create the *differential*

image. Finally, we integrate the differential image, and the left/right view of the DP image pair is given by

$$\hat{\mathbf{B}}_{\{L,R\}} = \mathcal{T}(\mathcal{I}_{\{L,R\}}), \tag{19}$$

where $\mathcal{T}(\cdot)$ denotes the integral process. The integration process [52] is also closely related to 'summed-area tables' in graphics [50].

Our simulator enables the vision community to collect large amounts of DP data with ground truth, which opens the door to accurately benchmark DP-based methods. Furthermore, our simulator can be used to build a reblur loss that regularizes our depth estimate and image deblurring in the training process.

In our previous work, we build our DDDNet considering both stereo and defocus cues (seeing in Section 4.1) in a supervised manner. In this paper, we explicitly derive the backward gradients of the simulator to enable end-to-end training. We further implement the simulator, namely the Reblur module, in CUDA with pixel extrapolation in continuous space and pixel-level parallelism efficiently (seeing in Table 1). With the Reblur module, we turn all-in-focus images into supervisory signals for the unsupervised depth estimation and therefore propose our WDDNet (seeing in Section 4.2).

4 DP-based Depth and Deblur Network

In this section, we first introduce our previous work DDDNet in Section 4.1. Then, we introduce our WDDNet in Section 4.2 to jointly estimate the depth and restore the all-in-focus image.

4.1 DDDNet.

4.1.1 Network architecture.

The input of our DDDNet is the left and right view of a DP image pair $\mathbf{B}_{\{L,R\}}$. The output of our DDDNet is the estimated inverse depth map $\hat{\mathbf{D}}$ and the deblurred image $\hat{\mathbf{I}}$. We use ground-truth latent sharp image $\mathbf{I}$ and inverse depth map $\mathbf{D}$ for training.

The pipeline of DDDNet is shown in Fig. 8. It consists of two components: DepthNet $g(\cdot)$ with parameters $\mathcal{G}$ and DeblurNet $f(\cdot)$ with parameters $\mathcal{F}$. The DepthNet is based on [53] and the DeblurNet is based on the multi-patch network [54]. Note that our approach is independent of the choices of $f(\cdot)$ and $g(\cdot)$ (*e.g.*, multi-scale and multi-patch architectures). We start with a coarse inverse depth map $\hat{\mathbf{D}}_c$ estimation by the DepthNet, where $\hat{\mathbf{D}}_c = g(\mathbf{B}_{\{L,R\}}; \mathcal{G})$. Then, we combine the coarse and blurred inverse depth map with $\mathbf{B}_{\{L,R\}}$, and feed it to the DeblurNet. The DeblurNet is an encoder-decoder network, and $\{\hat{\mathbf{I}}, \hat{\mathbf{D}}\} = f(\mathbf{B}_{\{L,R\}}, \hat{\mathbf{D}}_c; \mathcal{F})$.

4.1.2 Loss functions.

We use a combination of image restoration loss, depth loss, and image reblur loss. The final loss is the sum of the three losses.

$$\mathcal{L} = \mathcal{L}_{\text{res}} + \mathcal{L}_{\text{d}} + \mathcal{L}_{\text{reb}} \ . \tag{20}$$

Image restoration loss $\mathcal{L}_{\text{res}}$. This loss ensures the deblurred image is similar to the target image, and is given by

$$\mathcal{L}_{\text{res}} = \frac{1}{N} \sum_{y,z} \|\mathbf{I}(y,z) - \hat{\mathbf{I}}(y,z)\| \ , \tag{21}$$

where N is the number of pixels and $\|\cdot\|$ is the ℓ_2 norm.

Depth loss $\mathcal{L}_{\text{d}}$. We adopt the smooth ℓ_1 loss. Smooth ℓ_1 loss $\mathcal{S}(\cdot)$ is widely used in a regression task, for its robustness and low sensitivity to outliers [55]. The $\mathcal{L}_{\text{d}}$ is defined as

$$\mathcal{L}_{\text{d}} = \frac{1}{N} \sum_{y,z} \mathcal{S}(\mathbf{D}(y,z) - \hat{\mathbf{D}}(y,z)) \ . \tag{22}$$

Image reblur loss $\mathcal{L}_{\text{reb}}$. The reblur loss penalizes the differences between the input blurred images and the reblurred images (from the DP simulator). The reblur loss explicitly enforces the restored image and inverse depth map to lie on the manifold of the ground-truth image and inverse depth map, subject to our DP model. The $\mathcal{L}_{\text{reb}}$ is given by

$$\mathcal{L}_{\text{reb}} = \frac{1}{N} \sum_{y,z} \|\mathbf{B}_{\{L,R\}}(y,z) - \hat{\mathbf{B}}_{\{L,R\}}(y,z)\| \ . \tag{23}$$

4.2 WDDNet.

4.2.1 Network Architecture

The pipeline of our method is given in Fig. 2. The inputs of our model are left and right views of the DP pair, $\mathbf{B}_L$ and $\mathbf{B}_R$. The outputs of our model are the estimated inverse depth map $\hat{\mathbf{D}}$ and the restored all-in-focus image $\hat{\mathbf{I}}$.

Our model consists of three components: the DAiF module $f(\cdot)$ (Sec. 4.2.2), the Reblur module $m(\cdot)$ (Sec. 4.2.3), and the Fstack module $s(\cdot)$ (Sec. 4.2.4). Start with the DP pair $\mathbf{B}_L$ and $\mathbf{B}_R$, the DAiF module outputs an all-in-focus image and an inverse depth map, which is $\{\hat{\mathbf{I}}, \hat{\mathbf{D}}\} = f(\mathbf{B}_L, \mathbf{B}_R)$. The GT all-in-focus image $\mathbf{I}$ is used as the supervision signal to ensure the correctness of $\hat{\mathbf{I}}$. Then, the depth map $\hat{\mathbf{D}}$ and image $\hat{\mathbf{I}}$ are fed to the Reblur module to generate the reblurred DP pair $\hat{\mathbf{B}}_{\{L,R\}}$, which is $\hat{\mathbf{B}}_{\{L,R\}} = m(\hat{\mathbf{I}}, \hat{\mathbf{D}})$. The input DP pair is used to supervise the reblurred DP pair. Furthermore, by varying the focal distance F', a focal stack can be obtained, which is $\hat{\mathbf{S}}_{\{L,R\}} = s(\hat{\mathbf{I}}, \hat{\mathbf{D}}, F')$. The supervision signal of $\hat{\mathbf{S}}_{\{L,R\}}$ is the 'GT' focal stack, $\mathbf{S}_{\{L,R\}} = s(\mathbf{I}, \hat{\mathbf{D}}, F')$.

4.2.2 The DAiF module

As shown in Fig. 1 (first column), the DP blur is always accompanied by a shift between left and right DP images. This is due to light rays from the object $\mathbf{Y}$ within the DoF converging at a single-pixel unit $\mathbf{Y}'$ on the image plane, resulting in an in-focus pixel and zero-disparity between left and right DP views. In the meantime, light rays from a point $\mathbf{X}$ outside the DoF region spread to DP blur regions, producing a DP disparity/inverse depth between left and right DP views. In summary, left and right images show larger intensity differences in blurred regions than non-blurred regions due to DP blur and DP disparity.

Therefore, we use a self-attention mechanism that follows the nature of the DP pair to pay more attention to DP blurred regions. Specifically, i) we use an encoder module to extract features $\mathcal{S}_1$ from $|\mathbf{B}_L - \mathbf{B}_R|$, and $\mathcal{S}_2$ from the concatenation of $\mathbf{B}_L$ and $\mathbf{B}_R$; ii) we calculate a spatial attention map $\mathcal{A}$ from $\mathcal{S}_1$ using [56]; iii) $\mathcal{S}_2 + \mathcal{A}$ is fed to a decoder module to estimate $\hat{\mathbf{I}}$; and iv) $\hat{\mathbf{I}}$, $\mathbf{B}_L$, and $\mathbf{B}_R$ are fed to the cost-volume base network architecture [53] to estimate $\hat{\mathbf{D}}$.

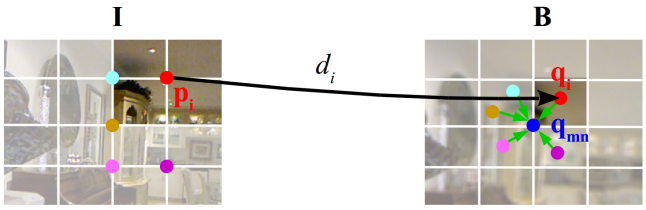


Fig. 9. *Illustration of pixel extrapolation. Discrete pixel $\mathbf{p}_i$ in $\mathbf{I}$ is projected to continuous $\mathbf{q}_i$ in $\mathbf{B}$ with its depth d_i. The pixel value of $\mathbf{q}_{mn}$ is then computed by weighted-averaging the pixel values in its neighbours, and the weight is defined in Eq. (25).*

4.2.3 The Reblur module

Our Reblur model, *i.e.* the simulator in Fig. 8, maps points in $\mathbf{I}$ to two point sets in $\mathbf{B}_L$ and $\mathbf{B}_R$ based on the scene depth as in Eq. (15) and Eq. (18).

Specifically, i) for an integer-valued pixel $\mathbf{p}_i = (x_{\mathbf{p}_i}, y_{\mathbf{p}_i})$ in $\mathbf{I}$, we first compute its corresponding pixel $\mathbf{q}_i = (x_{\mathbf{q}_i}, y_{\mathbf{q}_i})$ in $\mathbf{B}$ (either $\mathbf{B}_L$ or $\mathbf{B}_R$), using Eq. (24); ii) we assign the intensity of pixel $\mathbf{p}_i$ to $\mathbf{q}_i$, *i.e.* $\mathbf{I}(\mathbf{p}_i) = \mathbf{B}(\mathbf{q}_i)$. Collecting all pixel values in $\mathbf{B}$ yields a set $\{\mathbf{B}(\mathbf{q}_i)|i = 1, 2, ..., T\}$, where T is the total number of pixels in $\mathbf{I}$; iii) For each integer-valued pixel $\mathbf{q}_{mn}$ in $\mathbf{B}$, we compute its K ($K = 4$) nearest neighbours, with respect to the real-valued set $\{\mathbf{q}_i|i = 1, 2, ..., T\}$; iv) The pixel value of $\mathbf{q}_{mn}$ is computed by weighted-averaging the pixel values in its K-nearest neighbours. Please refer to Fig. 9 for more details. The above process is summarized into the following equations:

$$(x_{\mathbf{q}_i}, y_{\mathbf{q}_i}) = \left(1 - \frac{F}{f} + \frac{F}{d_i}\right)(x_c, y_c) + \frac{F}{f}(x_{\mathbf{p}_i}, y_{\mathbf{p}_i}) \ , \quad (24)$$

$$g(\mathbf{q}_i, \mathbf{q}_{mn}) = (1 - |x_{\mathbf{q}_i} - x_{\mathbf{q}_{mn}}|)\,(1 - |y_{\mathbf{q}_i} - y_{\mathbf{q}_{mn}}|) \ , \quad (25)$$

$$\mathbf{B}(\mathbf{q}_{mn}) = \frac{\sum_{\mathbf{q}_i \in \mathcal{E}(\mathbf{q}_{mn})} g(\mathbf{q}_i, \mathbf{q}_{mn})\mathbf{B}(\mathbf{q}_i)}{\sum_{\mathbf{q}_i \in \mathcal{E}(\mathbf{q}_{mn})} g(\mathbf{q}_i, \mathbf{q}_{mn})} \ , \quad (26)$$

where $\mathbf{q}_i$ is real-valued and $\mathbf{q}_{mn}$ is integer-valued, respectively. $\mathcal{E}(\mathbf{q}_{mn})$ denotes the K nearest neighbor of $\mathbf{q}_{mn}$, and (x_c, y_c) is a point in either $\mathbf{A}_L$ or $\mathbf{A}_R$. We then derive the gradients of loss $\mathcal{L}$ over depth d_i for $\mathbf{q}_i$ as follows,

$$\begin{aligned}\nabla_{d_i}\mathcal{L} &= \frac{\partial \mathcal{L}}{\partial \mathbf{B}(\mathbf{q}_{mn})}\frac{\partial \mathbf{B}(\mathbf{q}_{mn})}{\partial d_i} \\ &= \frac{\partial \mathcal{L}}{\partial \mathbf{B}(\mathbf{q}_{mn})}\frac{\partial \mathbf{B}(\mathbf{q}_{mn})}{\partial g(\mathbf{q}_i, \mathbf{q}_{mn})}\frac{\partial g(\mathbf{q}_i, \mathbf{q}_{mn})}{\partial d_i} \\ &= \frac{\partial \mathcal{L}}{\partial \mathbf{B}(\mathbf{q}_{mn})}\frac{\mathbf{B}(\mathbf{q}_i) - \mathbf{B}(\mathbf{q}_{mn})}{\sum\limits_{\mathbf{q}_i \in \mathcal{E}(\mathbf{q}_{mn})} g(\mathbf{q}_i, \mathbf{q}_{mn})} \\ &\qquad \left(\frac{\partial g(\mathbf{q}_i, \mathbf{q}_{mn})}{\partial x_{\mathbf{q}_i}}\frac{\partial x_{\mathbf{q}_i}}{\partial d_i} + \frac{\partial g(\mathbf{q}_i, \mathbf{q}_{mn})}{\partial y_{\mathbf{q}_i}}\frac{\partial y_{\mathbf{q}_i}}{\partial d_i}\right).\end{aligned} \quad (27)$$

The feasibility of Eq. (27) relies on $g(\mathbf{q}_i, \mathbf{q}_{mn})$, which is a function of d_i.

Then, we detail the gradient derivation of loss $\mathcal{L}$ over depth d_i from all surrounding pixels of $\mathcal{E}(\mathbf{q}_i)$, denoted as $\{\mathbf{q}_k\}$ after replacing $\mathbf{q}_i$ in Eq. (26) for a better readability. Remember that $\mathbf{q}_i$ and $\mathbf{q}_k$ have real-valued coordinates and $\mathbf{q}_{mn}$ has integral coordinates by rounding $\mathbf{q}_i$ to 4 nearest pixels.

$$\begin{aligned}\nabla^*_{d_i}\mathcal{L} &= \sum_{\mathbf{q}_{mn} \in \mathcal{E}(\mathbf{q}_i)} \nabla_{d_i}\mathcal{L} \\ &= \sum_{\mathbf{q}_{mn} \in \mathcal{E}(\mathbf{q}_i)} \frac{\partial \mathcal{L}}{\partial \mathbf{B}(\mathbf{q}_{mn})}\frac{\partial \mathbf{B}(\mathbf{q}_{mn})}{\partial d_i} \\ &= \sum_{\mathbf{q}_{mn} \in \mathcal{E}(\mathbf{q}_i)} \frac{\partial \mathcal{L}}{\partial \mathbf{B}(\mathbf{q}_{mn})} \sum_{\mathbf{q}_k \in \mathcal{E}(\mathbf{q}_{mn})} \frac{\partial \mathbf{B}(\mathbf{q}_{mn})\partial g(\mathbf{q}_k, \mathbf{q}_{mn})}{\partial g(\mathbf{q}_k, \mathbf{q}_{mn})\partial d_i} \ .\end{aligned} \quad (28)$$

Since $\partial g(\mathbf{q}_k, \mathbf{q}_{mn})/\partial d_i$ is non-zero only when $k = i$, we have

$$\begin{aligned}\nabla^*_{d_i}\mathcal{L} &= \sum_{\mathbf{q}_{mn} \in \mathcal{E}(\mathbf{q}_i)} \frac{\partial \mathcal{L}}{\partial \mathbf{B}(\mathbf{q}_{mn})}\frac{\partial \mathbf{B}(\mathbf{q}_{mn})}{\partial g(\mathbf{q}_i, \mathbf{q}_{mn})}\frac{\partial g(\mathbf{q}_i, \mathbf{q}_{mn})}{\partial d_i} \\ &= \sum_{\mathbf{q}_{mn} \in \mathcal{E}(\mathbf{q}_i)} \frac{\partial \mathcal{L}}{\partial \mathbf{B}(\mathbf{q}_{mn})}\frac{\mathbf{B}(\mathbf{q}_i) - \mathbf{B}(\mathbf{q}_{mn})}{\sum_{\mathbf{q}_k \in \mathcal{E}(\mathbf{q}_{mn})} g(\mathbf{q}_k, \mathbf{q}_{mn})} \\ &\qquad \left(\frac{\partial g(\mathbf{q}_i, \mathbf{q}_{mn})}{\partial h_{\mathbf{q}_i}}\frac{\partial h_{\mathbf{q}_i}}{\partial d_i} + \frac{\partial g(\mathbf{q}_i, \mathbf{q}_{mn})}{\partial w_{\mathbf{q}_i}}\frac{\partial w_{\mathbf{q}_i}}{\partial d_i}\right) \ ,\end{aligned} \quad (29)$$

where

$$\frac{\partial g(\mathbf{q}_i, \mathbf{q}_{mn})}{\partial h_{\mathbf{q}_i}} = \begin{cases} -1 + |w_{\mathbf{q}_i} - w_{\mathbf{q}_{mn}}| \ , & \text{if } w_{\mathbf{q}_i} > w_{\mathbf{q}_{mn}} \\ 1 - |w_{\mathbf{q}_i} - w_{\mathbf{q}_{mn}}| \ , & \text{otherwise} \end{cases} \quad (30)$$

and

$$\frac{\partial h_{\mathbf{q}_i}}{\partial d_i} = \frac{\partial}{\partial d_i}\left(1 - \frac{F}{f} + \frac{F}{d_i}\right)x_c + \frac{F}{f}h_{\mathbf{p}_i} = -\frac{F x_c}{d_i^2} \ . \quad (31)$$

Similarly, one can calculate $\partial g(\mathbf{q}_i, \mathbf{q}_{mn})/\partial w_{\mathbf{q}_i}$ and $\partial w_{\mathbf{q}_i}/\partial d_i$ for Eq. (29).

Our implementation in CUDA accelerates the process, shown in Table 1.

TABLE 1
Efficiency and precision of our Reblur module. 'MRE': maximum relative error. Time and error are averaged on 1000 random samples.

Simulation (8×3×32×64)	Forward Time	Forward MRE	Backward Time	Backward MRE
PyTorch GPU	75s	0	219s	0
Ours	0.41ms	1.5e^{-4}	0.08ms	3.9e^{-3}

4.2.4 The Fstack module

In our network, we use GT $\mathbf{I}$ as the supervisory signal for unsupervised depth estimation and the qualities of $\hat{\mathbf{I}}$ and $\hat{\mathbf{D}}$ are jointly improved in the training process. To further improve the quality of $\hat{\mathbf{I}}$, we use the Fstack module to regularize our network at each focal plane.

First, we change the focal distance from F to $F' = F + \Delta$. From Eq. (17), $\hat{\mathbf{D}}$ is then become to $\lambda_1\hat{\mathbf{D}} + \lambda_2$, where $\lambda_1 = 1 - \Delta/F$ and $\lambda_2 = -\Delta b/F$ (see in the supplementary material). By varying F', we can get a group of $\{\hat{\mathbf{D}}_n = \lambda_1^n\hat{\mathbf{D}} + \lambda_2^n\}$. Then, the Fstack module maps $\hat{\mathbf{I}}$ to a sequence of images with $\{\hat{\mathbf{D}}_n\}$, which is $\hat{\mathbf{S}}_{\{L,R\}} = s(\hat{\mathbf{I}}, \hat{\mathbf{D}}, F') = \{m(\hat{\mathbf{I}}, \hat{\mathbf{D}}_1), \cdots, m(\hat{\mathbf{I}}, \hat{\mathbf{D}}_n)\}$. The supervision signal of $\hat{\mathbf{S}}_{\{L,R\}}$ is the 'GT' focal stack, which is $\mathbf{S}_{\{L,R\}} = \{m(\mathbf{I}, \hat{\mathbf{D}}_1), \cdots, m(\mathbf{I}, \hat{\mathbf{D}}_n)\}$.

Same as the Reblur module, the Fstack modules is also based on the DP model and is implemented in CUDA with pixel extrapolation and pixel-level parallelism (see Sec. 4.2.3).

4.2.5 Loss Functions.

We use a combination of an image restoration loss, an image reblur loss and a depth loss. The final loss is the sum of the three losses.

$$\mathcal{L} = \mathcal{L}_{\text{res}} + \mathcal{L}_{\text{reb}} + \lambda \mathcal{L}_{\text{dep}} \ , \tag{32}$$

where λ is a weight parameter, we set $\lambda = 10^{-3}$ in all our experiments [36].

Image restoration loss $\mathcal{L}_{\text{res}}$. This loss ensures the deblurred image is similar to the target (latent sharp) image.

$$\mathcal{L}_{\text{res}} = \mathcal{L}_{\text{MSE}}(\hat{\mathbf{I}}, \mathbf{I}) + \mathcal{L}_{\text{LPIPS}}(\hat{\mathbf{I}}, \mathbf{I}) \ , \tag{33}$$

where the $\mathcal{L}_{\text{MSE}}$ is the MSE loss (ℓ_2 norm) between the output estimated $\hat{\mathbf{I}}$ and its ground truth $\mathbf{I}$. Moreover, we use a perceptual loss $\mathcal{L}_{\text{LPIPS}}$ (the calibrated perceptual loss [60]). The perceptual loss passes the reconstructed image and the target image through a VGG network [61] that was trained on ImageNet [62], and averages the distances between VGG features across multiple layers. By minimizing $\mathcal{L}_{\text{LPIPS}}$, the network effectively learns to endow the deblurred image with natural statistics (*i.e.*, with features close to those of natural images).

Image reblur loss $\mathcal{L}_{\text{reb}}$. This loss contains a $\mathcal{L}_{\text{b}}$ loss and a $\mathcal{L}_{\text{f}}$ loss. The $\mathcal{L}_{\text{b}}$ loss penalizes the differences between the input DP pair and the reblurred DP pair. The $\mathcal{L}_{\text{f}}$ loss penalizes the difference between the focal stack that is based on $\hat{\mathbf{I}}$ and $\hat{\mathbf{D}}$, and the one that is based on $\mathbf{I}$ and $\hat{\mathbf{D}}$. The $\mathcal{L}_{\text{reb}}$ loss allows us to explicitly enforce the solution to lie on the manifold of the sharp image and depth map that explains the observed DP images.

$$\begin{aligned} \mathcal{L}_{\text{b}} &= \mathcal{L}_{\text{MSE}}(\hat{\mathbf{B}}_{\{L,R\}}, \mathbf{B}_{\{L,R\}}) + \mathcal{L}_{\text{LPIPS}}(\hat{\mathbf{B}}_{\{L,R\}}, \mathbf{B}_{\{L,R\}}) \ , \\ \mathcal{L}_{\text{f}} &= \mathcal{L}_{\text{MSE}}(\hat{\mathbf{S}}_{\{L,R\}}, \mathbf{S}_{\{L,R\}}) + \mathcal{L}_{\text{LPIPS}}(\hat{\mathbf{S}}_{\{L,R\}}, \mathbf{S}_{\{L,R\}}) \ . \end{aligned} \tag{34}$$

Depth loss $\mathcal{L}_{\text{dep}}$. We encourage the depth map to be locally smooth by using an L1 penalty term $\mathcal{L}_{\text{s}}$ on the depth gradients [36] and a perception loss [60] $\mathcal{L}_{\text{p}}$ (Alex feature [63]) for depth map.

$$\mathcal{L}_{\text{dep}} = \mathcal{L}_{\text{s}}(\hat{\mathbf{D}}, \mathbf{I}) + \mathcal{L}_{\text{p}}(\hat{\mathbf{D}}, \mathbf{I}) \ . \tag{35}$$

5 Experiment

5.1 Experimental setup

Synthetic dataset. We evaluate our method on two synthetic DP datasets, namely, the DDD-syn [15] and the RPDP [18] dataset. The DDD-syn dataset contains 5,000 pairs for training and 500 pairs for testing. Each pair contains an all-in-focus image, a simulated DP pair and its associated ground-truth (GT) depth map. The simulation is based on the NYU depth dataset [64]. To simulate a DP pair, the 'integral image' is used to approximate the point spread function. The RPDP dataset synthesizes 2,023 training and 201 testing DP pairs based on SYNTHIA [65]. Each pair contains an all-in-focus image and a simulated DP pair. Different from DDD-syn, they define a parametric point spread function model based on the Butterworth filter to simulate the DP blur.

Real dataset. We also evaluate our method on two real DP datasets, namely, the Defocus Deblur Dual-Pixel dataset (DPD-blur) [10] and the Defocus Depth estimation dataset (DPD-disp) [6]. The DPD-blur dataset is captured using a Canon EOS 5D Mark IV DSLR camera. DP pairs and their associated all-in-focus images are provided (351 training and 76 testing pairs). GT depth maps are not available. The DPD-disp dataset provides DP pairs with GT depth maps (100 testing pairs). Canon captures a DP pair, and its associated depth map is computed by the widely used HeliconSoft software with the captured focal stacks. Note, this dataset does not provide training sets, thus all methods[4] are directly tested on it without training. Furthermore, this dataset does not provide all-in-focus images. Therefore, our pre-trained model (using the DPD-blur dataset) is also directly tested on the dataset.

Evaluation metrics. Given a DP pair, our method jointly estimates a depth map and an all-in-focus image. For the depth map, we use absolute relative error 'abs_rel', square relative error 'sq_rel', root mean square error 'rmse' and its log scale 'rmse_log', and the δ inlier ratios (maximal mean relative error of $\delta_i = 1.25^i$ for $i \in 1, 2, 3$). Note, the baseline is provided by the dataset. For the restored image, we adopt the peak signal-to-noise ratio 'PSNR', structural similarity 'SSIM', and RMSE relative error 'RMSE_rel' that explains what 1dB improvement means [15]. For the DPD-disp dataset, we follow DPdisp [6] to use the affine invariant version of MAE ('AI(1)'), RMSE ('AI(2)'), Spearman's rank correlation ('$1 - |\rho_s|$') and Geometric Mean ('GM') for evaluation.

Baseline methods. For the depth map, we comparing with the SOTA monocular (LeRes [59], BTS [35], Mono2 [36]), stereo (HITNet [58], AnyNet [29]), and DP (SDoF [14], DPdisp [6], MPI [8] DDDNet [15]) based depth estimation methods. For defocus deblurring, we compared with EBDB [57], DMENet [45], DPDNet [10], DDDNet [15], and RDPD [18]. For learning-based methods, we directly use their pre-trained models, fine-tune them, or re-train them, and we try to report the best results. All learning-based methods are trained (except on the DPD-disp dataset) and evaluated on each of the above four datasets independently.

Implementation details. Our WDDNet network is trained for 100 epochs from scratch using the Adam optimizer [66] with a learning rate of 10^{-4}. We randomly crop a 192×192 patch from the original image for batch training, and we use a batch size of 16. Following common practices, we also randomly augment patches by adjusting their brightness in the training stage. Our model is trained on four NVIDIA Tesla P100 GPU.

5.2 Results

On the above four datasets, we compare our method with SOTA methods on depth estimation and image deblurring.

Deblurring results. We provide deblurring comparisons on the DDD-syn [15], RDPD [18], and DPD-blur [10] datasets due to their GT all-in-focus images.

4. Except DPdisp [6], they have access to the training set of the DPD-disp dataset and use it for training.

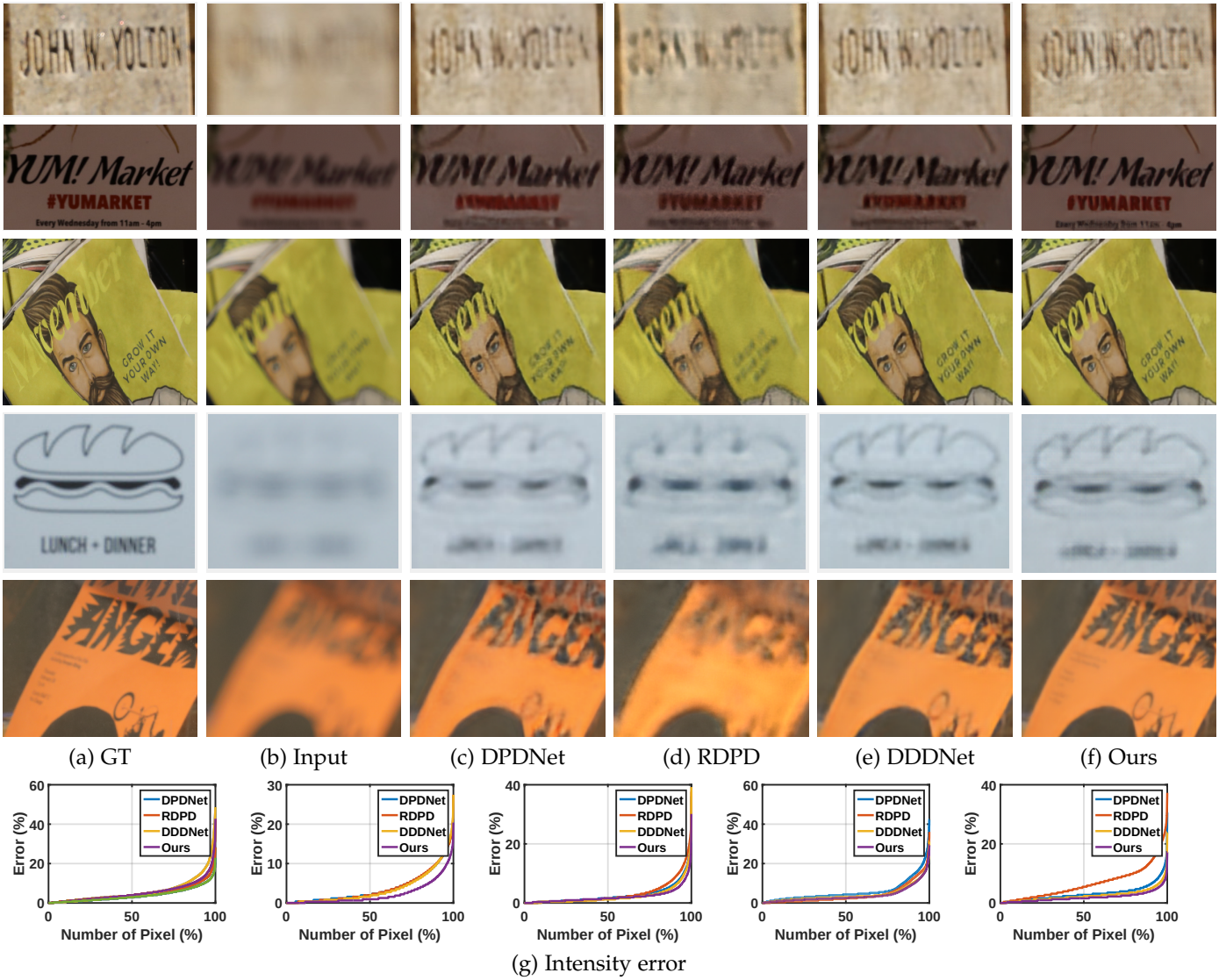


Fig. 10. *Comparisons of image deblurring on the DPD-blur dataset [10]. (a) GT all-in-focus image. (b) The input DP image (left-view). (c) Deblurred by DPDNet [10]. (d) Deblurred by RPDP [18]. (e) Deblurred by DDDNet [15]. (f) Ours. (g) The distribution of the intensity error ('RMSE_rel'). It shows the percentile of the number of pixels below an error ratio. The lower the curve, the better. Our method outperforms other baselines.*

The results on the DDD-syn [15] dataset are given in Table 2. The performance of our method outperforms other SOTA methods. The PSNR, SSIM, and RMSE_rel of our method and the second-best DDDNet are $35.64/0.957/1.65$ and $33.21/0.956/2.17$, respectively. This demonstrates that our model, with a focal stack training scheme, significantly improves the deblurring performance.

The results on the RDPD [18] dataset are given in Table 2. It also shows that our method has the best performance. The PSNR, SSIM, and RMSE_rel of our method and the second-best RDPD are $32.64/0.861/2.33$ and $31.09/0.861/2.79$, respectively. Unlike DDD-syn, DP pairs from the RDPD dataset are simulated by using a different method (Butterworth filter). Despite different simulation methods, our DDD-sys trained model achieves strong performance on the RDPD dataset. This indicates the generalization ability of our model.

The results on the DPD-blur [10] dataset are given in Table 2. We again achieve the best performance. The PSNR, SSIM, and RMSE_rel of our method and the second-best DDDNet are $25.86/0.856/5.09$ and $25.41/0.839/5.36$, respectively. Some qualitative comparisons are given in Fig. 10.

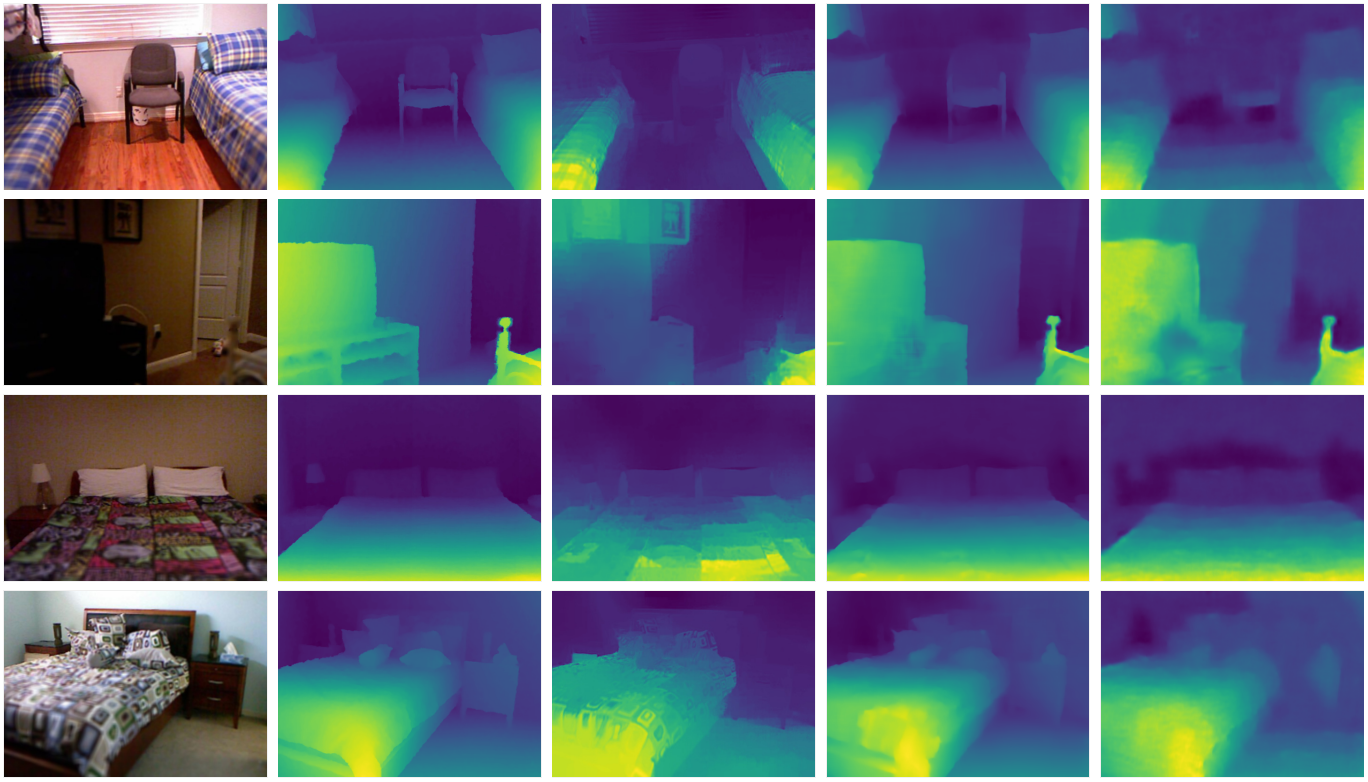
Depth results. We provide depth comparisons on the DDD-syn [15] and DPD-disp datasets [6] since the two datasets have GT depth maps. The results on the DDD-syn dataset [15] are shown in Table 3 and Fig. 11. Our weakly-supervised model ('Ours') achieves the second-best performance among all SOTA DP-based, monocular, and stereo approaches. The 'abs_rel'/'sq_rel'/'rmse' of our method and the best supervised method DDDNet are $0.109/0.093/0.580$ and $0.83/0.052/0.461$, respectively.

The results on the DPD-disp dataset [6] are given in Table 4 and Fig. 12. Note, all methods (except DPdisp [6]) are directly tested on DPD-disp without training. Our model ('Ours') achieves competitive performance compared with the SOTA methods [6], [14], [15], [35], [36], [58], [59].

To further illustrate the superiority of our results, we reconstruct the point cloud for the comparison. An example is shown in Fig. 13. Then, we reconstruct the scene to other views with the estimated sharp image and depth map. The reconstructed videos can be found in our project page.

TABLE 2
Comparison of deblurring results on the DDD-syn dataset [15], the RDPD dataset [18], and the DPD-blur dataset [10]. We highlight the best and the second-best numbers in red and green, respectively.

Dataset		EBDB [57]	DMENet [45]	DPDNet [10]	RDPD [18]	DDDNet [15]	IFAN [47]	Ours
DDD-syn	PSNR ↑	26.48	30.14	31.45	32.42	33.21	34.18	35.64
	SSIM ↑	0.683	0.939	0.926	0.912	0.956	0.929	0.957
	RMSE_rel ↓	4.74	3.11	2.68	2.39	2.17	1.95	1.65
RDPD	PSNR ↑	24.23	25.17	29.84	31.09	30.14	31.12	32.64
	SSIM ↑	0.637	0.731	0.828	0.861	0.840	0.865	0.861
	RMSE_rel ↓	6.14	5.51	3.22	2.79	3.11	2.78	2.33
DPD-blur	PSNR ↑	23.45	23.55	25.13	25.39	25.41	25.99	25.86
	SSIM ↑	0.683	0.720	0.786	0.772	0.839	0.804	0.856
	RMSE_rel ↓	6.72	6.65	5.54	5.38	5.36	5.01	5.09



(a) Image (b) GT (c) DPdisp (d) DDDNet (e) Ours

Fig. 11. *Comparison of depth estimation results on the DDD-syn [15] dataset. (a) The input DP image (left-view). (b) The GT depth map. (c) Depth estimated by DPdisp [6]. (d) Depth estimated by DDDNet [15]. (e) Our weakly-supervised depth estimation results.*

5.3 Discussions

The usefulness of the weakly-supervised model. Though our model is trained in a weakly-supervised manner, readers may ask could the proposed method benefit from 1) a small number of labelled GT data (DP pair with depth map) and 2) a large number of unlabelled data (DP pair with all-in-focus image)? To answer the two questions, we separately conduct two depth estimation experiments on the DDD-syn [15] and DPD-disp datasets [6]: 1) feed a small number of GT data to further fine-tune our model and 2) use more unlabelled data to further fine-tune our model.

For the first experiment, we only use 10% (500/5,000 images) labelled data (DP pairs, all-in-focus images, and depth maps) from the DDD-sys training set to fine-tune our pre-trained weakly-supervised model (obtained on DDD-sys). The results are given in Table 5. Our model ('Ours+') achieves the best performance in 50 epochs. Furthermore, if we use all labelled data, the performance of our method ($\text{Ours}_{\text{full}}$) can be further improved. The 'abs_rel'/'sq_rel'/'rmse_log'/ achieves $0.067/0.027/0.087$ in 10 epochs.

For the second experiment, we use unlabeled data (DP pairs and all-in-focus images) from the RDPD dataset to fine-tune our pre-trained weakly-supervised model (obtained on DPD-blur). The results on the DPD-disp datasets [6] are given in Table 6. It shows that the performance of our method can be improved. For example, in terms of the AI(1) metric, increases from 0.0773 to 0.0552.

The usefulness of our weakly-supervised model is not limited to depth estimations. Besides the improved depth quality for the above two experiments, we also observe improved image quality. For example, 1) on the DDD-syn dataset, the PSNR/SSIM/RMSE_rel of our weakly-supervised model and supervised model from the

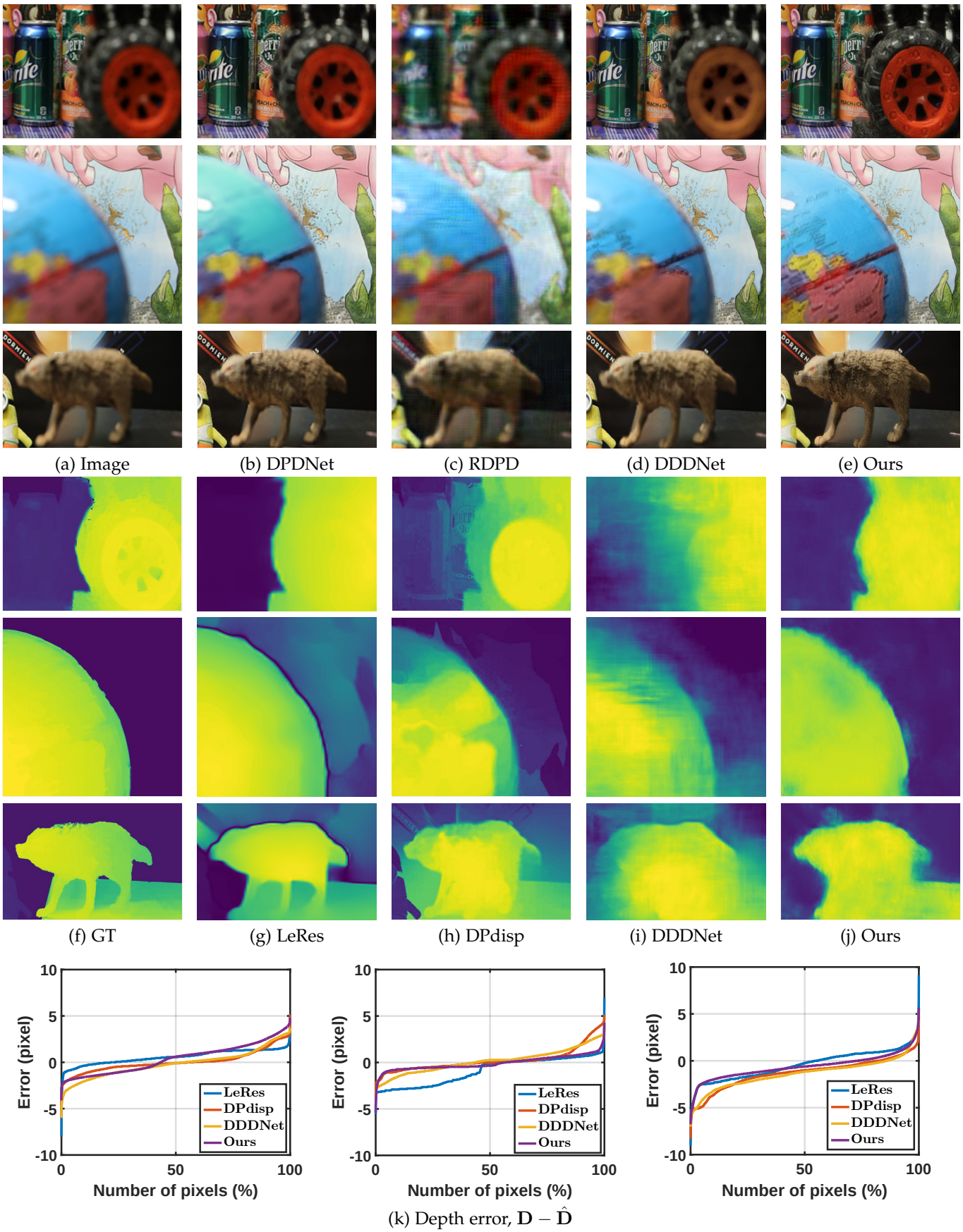


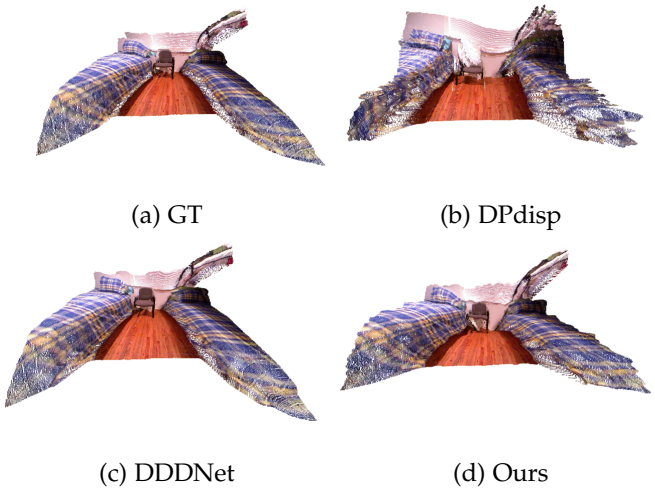
Fig. 12. *Comparisons of depth estimation and image deblurring on the DPD-disp dataset [6]. (a) The input DP image (left-view). (b) Deblurred by DPDNet [10]. (c) Deblurred by RDPD [18]. (d) Deblurred by DDDNet [15]. (e) Our deblurring results. (f) The ground truth depth map. (g) Depth estimated by LeRes [59]. (h) Depth estimated by DPdisp [6]. (i) Depth estimated by DDDNet [15]. (j) Our weakly-supervised depth estimation results. (k) Inverse depth error: $\mathbf{D} - \hat{\mathbf{D}}$, the closer to zero, the better. The error distributions help to statistically analyze the accuracy of each method. Our method achieves competitive performance compared with the SOTA supervised methods. (Best viewed in colour on screen.)*

TABLE 3
Comparison of depth estimations on the DDD-sys synthetic dataset [15]. The best numbers are highlighted in red. The second-best results are highlighted in green. Our weakly-supervised model ('Ours') achieves competitive performance compared with SOTA supervised approaches.

Sensor	Method	abs_rel ↓	sq_rel ↓	rmse ↓	rmse_log ↓	$a < 1.25 \uparrow$	$a < 1.25^2 \uparrow$	$a < 1.25^3 \uparrow$
Stereo	AnyNet [29]	0.128	0.117	0.731	0.168	0.828	0.923	0.998
	HITNet [58]	0.464	0.725	1.62	0.711	0.096	0.302	0.302
Monocular	LeRes [59]	0.330	0.393	1.123	0.479	0.280	0.506	0.781
	BTS [35]	0.377	0.195	0.255	0.484	0.554	0.741	0.994
Dual-pixel	DPdisp [6]	0.328	0.479	1.252	0.332	0.438	0.804	0.965
	MPI [8]	0.273	0.627	1.834	0.354	0.548	0.830	0.920
	DDDNet [15]	0.083	0.052	0.461	0.111	0.936	0.998	1.000
	Ours	0.109	0.093	0.580	0.141	0.915	0.987	0.994

TABLE 4
Comparison of depth estimations on the DPD-disp dataset [6]. The DPD-disp dataset only provides testing data and methods (except DPdisp) are directly tested using pre-trained models. The numbers of methods DPD-disp, SDoF, and Mono2 are borrowed from DPdisp [6]

	BTS [35]	Mono2 [36]	HITNet [58]	LeRes [59]	SDoF [14]	MPI [8]	DPdisp [6]	DDDNet [15]	Ours
AI(1) ↓	0.1070	0.1139	0.1289	0.0945	0.0875	0.1174	0.0481	0.0609	0.0773
AI(2) ↓	0.1767	0.1788	0.1938	0.1423	0.1294	0.1717	0.0743	0.0985	0.1137
1-ρ_s ↓	0.6149	0.6153	0.7748	0.2751	0.2910	0.7139	0.0779	0.1026	0.1933
GM ↓	0.2686	0.2285	0.3658	0.1706	0.1443	0.2433	0.0645	0.1098	0.2718



(a) GT (b) DPdisp

(c) DDDNet (d) Ours

Fig. 13. *An example of reconstructed point clouds on the DDD-syn dataset [15]. (a) The ground truth point cloud (by the ground truth depth map and all-in-focus image). (b) The results of DPdisp [6]. (c) The results of DDDNet [15]. (d) Ours. (Best viewed in colour on screen.)*

first experiment ('Ours_full') are $35.64/0.957/1.65$ and $36.79/0.961/1.45$, respectively; 2) on the RDPD dataset, the PSNR/SSIM/RMSE_rel of our weakly-supervised model and supervised model from the second experiment ('Ours+') are $32.64/0.861/2.33$ and $33.31/0.946/2.16$, respectively.

Effectiveness of different loss functions. To show the effectiveness of different loss terms in Eq. (20), we use DDD-syn dataset and train three models by independently removing $\mathcal{L}_{\mathrm{p}}$ and $\mathcal{L}_{\mathrm{s}}$. The results are given in Table 7. It shows that our model can get the best performance when using all three losses.

6 Conclusions

In this paper, we derive a mathematical DP model formulating the imaging process of the DP camera. We propose an end-to-end **WDDNet** (Weakly-supervised Depth and Deblur Network) by using a more efficient reblur module to jointly estimate an inverse depth map and restore a sharp image from a blurred DP image pair. The reblur module turns all-in-focus images into supervisory signals for unsupervised depth estimation. We explicitly derive the backward gradients to enable end-to-end training. Extensive experimental evaluation on both synthetic and real data shows that our weakly-supervised model achieves competitive performance compared to state-of-the-art supervised depth estimation methods. We hope the proposed DP simulator and reblur loss will motivate future work.

TABLE 5
Our unsupervised model can benefit from a small number of labelled data (DP pairs, all-in-focus images, and depth maps). We only use 10% labelled data from the DDD-sys training set to finetune our pre-trained unsupervised model (obtained on DDD-sys). Our model ('Our+') achieves the best performance on the DDD-sys dataset [15]. When all labelled data are used, the performance of our model ('$\mathrm{Ours}_{\mathrm{full}}$') can be further improved.

Method	DDDNet [15]	Ours	Ours+	$\mathrm{Ours}_{\mathrm{full}}$
abs_rel ↓	0.083	0.109	0.078	0.067
sq_rel ↓	0.052	0.093	0.038	0.027
rmse ↓	0.461	0.580	0.365	0.329
rmse_log ↓	0.111	0.141	0.100	0.087
$a < 1.25 \uparrow$	0.936	0.915	0.977	0.989
$a < 1.25^2 \uparrow$	0.998	0.987	0.998	0.999
$a < 1.25^3 \uparrow$	1.000	0.994	0.999	1.000

TABLE 6
Our weakly-supervised model can benefit from more unlabelled data. We use unlabelled data from the RDPD training set (DP pair with all-in-focus image) to finetune our pre-trained model (obtained on DPD-blur). The result is reported as 'Ours+'. It achieves competitive performance on the DPD-disp dataset [6].

	AI(1) ↓	AI(2) ↓	1-ρ_s ↓	GM ↓
DPdisp [6]	0.0481	0.0743	0.0779	0.0645
DDDNet [15]	0.0609	0.0985	0.1026	0.1098
Ours	0.0773	0.1137	0.1933	0.2718
Ours+	0.0552	0.0710	0.0684	0.2781

TABLE 7
Effectiveness of different loss terms on the DDD-syn dataset [15]. 'Ours$_{\mathcal{L}_\mathrm{p}}$' denotes our model trained without $\mathcal{L}_\mathrm{p}$ loss, and the same for 'Ours$_{\mathcal{L}_\mathrm{s}}$' and 'Ours$_{\mathcal{L}_\mathrm{f}}$'. Our model achieves the best performance using all three losses.

Method	abs_rel ↓	sq_rel ↓	rmse ↓	rmse_log ↓	$a < 1.25$ ↑	$a < 1.25^2$ ↑	$a < 1.25^3$ ↑	PSNR ↑
Ours	0.109	0.093	0.580	0.141	0.915	0.987	0.994	35.64
Ours$_{\mathcal{L}_\mathrm{p}}$	0.738	4.496	3.241	0.569	0.356	0.656	0.815	-
Ours$_{\mathcal{L}_\mathrm{s}}$	0.438	0.832	1.526	0.402	0.429	0.679	0.867	-
Ours$_{\mathcal{L}_\mathrm{f}}$	0.147	0.250	0.802	0.201	0.796	0.975	0.988	34.08

ACKNOWLEDGMENTS

This research was supported in part by the Beijing Institute of Technology Research Fund Program for Young Scholars, BIT Special-Zone, National Natural Science Foundation of China (62302045), the (ARC) fellowship, and Discovery grants (DE180100628, DP200102274, DP190102261, DP220100800).

REFERENCES

[1] P. Śliwiński and P. Wachel, "A simple model for on-sensor phase-detection autofocusing algorithm," *Journal of Computer and Communications*, vol. 1, no. 06, p. 11, 2013.

[2] J. Jang, Y. Yoo, J. Kim, and J. Paik, "Sensor-based auto-focusing system using multi-scale feature extraction and phase correlation matching," *Sensors*, vol. 15, no. 3, pp. 5747–5762, 2015.

[3] C. Herrmann, R. S. Bowen, N. Wadhwa, R. Garg, Q. He, J. T. Barron, and R. Zabih, "Learning to autofocus," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2020, pp. 2230–2239.

[4] M. Choi, H. Lee, and H.-e. Lee, "Exploring positional characteristics of dual-pixel data for camera autofocus," in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, October 2023, pp. 13 158–13 168.

[5] R. Garg, N. Wadhwa, S. Ansari, and J. T. Barron, "Learning single camera depth estimation using dual-pixels," in *Proceedings of the IEEE International Conference on Computer Vision*, 2019, pp. 7628–7637.

[6] A. Punnappurath, A. Abuolaim, M. Afifi, and M. S. Brown, "Modeling defocus-disparity in dual-pixel sensors," in *IEEE International Conference on Computational Photography (ICCP)*, 2020.

[7] Y. Zhang, N. Wadhwa, S. Orts-Escolano, C. Häne, S. Fanello, and R. Garg, "Du^{2}net: Learning depth estimation from dual-cameras and dual-pixels," *arXiv preprint arXiv:2003.14299*, 2020.

[8] S. Xin, N. Wadhwa, T. Xue, J. T. Barron, P. P. Srinivasan, J. Chen, I. Gkioulekas, and R. Garg, "Defocus map estimation and deblurring from a single dual-pixel image," in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2021, pp. 2228–2238.

[9] D. Kim, H. Jang, I. Kim, and M. H. Kim, "Spatio-focal bidirectional disparity estimation from a dual-pixel image," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2023, pp. 5023–5032.

[10] A. Abuolaim and M. S. Brown, "Defocus deblurring using dual-pixel data," in *European Conference on Computer Vision*. Springer, 2020, pp. 111–126.

[11] Abdullah Abuolaim, M. Afifi, and M. S. Brown, "Improving single-image defocus deblurring: How dual-pixel images help through multi-task learning," in *IEEE/CVF Winter Conference on Applications of Computer Vision, WACV 2022, Waikoloa, HI, USA, January 3-8, 2022*. IEEE, 2022, pp. 82–90. [Online]. Available: https://doi.org/10.1109/WACV51458.2022.00016

[12] H. Yang, L. Pan, Y. Yang, R. Hartley, and M. Liu, "Ldp: Language-driven dual-pixel image defocus deblurring network," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024, pp. 24 078–24 087.

[13] A. Punnappurath and M. S. Brown, "Reflection removal using a dual-pixel sensor," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2019, pp. 1556–1565.

[14] N. Wadhwa, R. Garg, D. E. Jacobs, B. E. Feldman, N. Kanazawa, R. Carroll, Y. Movshovitz-Attias, J. T. Barron, Y. Pritch, and M. Levoy, "Synthetic depth-of-field with a single-camera mobile phone," *ACM Transactions on Graphics (TOG)*, vol. 37, no. 4, pp. 1–13, 2018.

[15] L. Pan, S. Chowdhury, R. Hartley, M. Liu, H. Zhang, and H. Li, "Dual pixel exploration: Simultaneous depth estimation and image restoration," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2021, pp. 4340–4349.

[16] Y. Yang, L. Pan, L. Liu, and M. Liu, "K3dn: Disparity-aware kernel estimation for dual-pixel defocus deblurring," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2023, pp. 13 263–13 272.

[17] P. Liang, J. Jiang, X. Liu, and J. Ma, "Bambnet: A blur-aware multi-branch network for dual-pixel defocus deblurring," *IEEE/CAA Journal of Automatica Sinica*, vol. 9, no. 5, pp. 878–892, 2022.

[18] A. Abuolaim, M. Delbracio, D. Kelly, M. S. Brown, and P. Milanfar, "Learning to reduce defocus blur by realistically modeling dual-pixel data," in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021, pp. 2289–2298.

[19] J. Hensley, T. Scheuermann, G. Coombe, M. Singh, and A. Lastra, "Fast summed-area table generation and its applications," in *Computer Graphics Forum*, vol. 24, no. 3. Citeseer, 2005, pp. 547–556.

[20] R. Fernando *et al.*, *GPU gems: programming techniques, tips, and tricks for real-time graphics*. Addison-Wesley Reading, 2004, vol. 590.

[21] R. A. Hamzah and H. Ibrahim, "Literature survey on stereo vision disparity map algorithms," *Journal of Sensors*, vol. 2016, 2016.

[22] J. Zbontar and Y. LeCun, "Stereo matching by training a convolutional neural network to compare image patches," *The journal of machine learning research*, vol. 17, no. 1, pp. 2287–2318, 2016.

[23] S. Duggal, S. Wang, W.-C. Ma, R. Hu, and R. Urtasun, "Deeppruner: Learning efficient stereo matching via differentiable patchmatch," in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, October 2019.

[24] L. Pan, Y. Dai, M. Liu, and F. Porikli, "Simultaneous stereo video deblurring and scene flow estimation," in *CVPR*, July 2017.

[25] L. Pan, Y. Dai, M. Liu, F. Porikli, and Q. Pan, "Joint stereo video deblurring, scene flow estimation and moving object segmentation," *IEEE Transactions on Image Processing*, vol. 29, pp. 1748–1761, 2019.

[26] F. Aleotti, F. Tosi, L. Zhang, M. Poggi, and S. Mattoccia, "Reversing the cycle: self-supervised deep stereo through enhanced monocular distillation," in *European Conference on Computer Vision*. Springer, 2020, pp. 614–632.

[27] N. Mayer, E. Ilg, P. Hausser, P. Fischer, D. Cremers, A. Dosovitskiy, and T. Brox, "A large dataset to train convolutional networks for disparity, optical flow, and scene flow estimation," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, pp. 4040–4048.

[28] A. Tonioni, F. Tosi, M. Poggi, S. Mattoccia, and L. D. Stefano, "Real-time self-adaptive deep stereo," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2019, pp. 195–204.

[29] Y. Wang, Z. Lai, G. Huang, B. H. Wang, L. Van Der Maaten, M. Campbell, and K. Q. Weinberger, "Anytime stereo image depth estimation on mobile devices," in *2019 International Conference on Robotics and Automation (ICRA)*. IEEE, 2019, pp. 5893–5900.

[30] H. Xu and J. Zhang, "Aanet: Adaptive aggregation network for efficient stereo matching," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2020, pp. 1959–1968.

[31] F. Liu, C. Shen, G. Lin, and I. Reid, "Learning depth from single monocular images using deep convolutional neural fields," *IEEE*

transactions on pattern analysis and machine intelligence, vol. 38, no. 10, pp. 2024–2039, 2015.

[32] H. Fu, M. Gong, C. Wang, K. Batmanghelich, and D. Tao, "Deep ordinal regression network for monocular depth estimation," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2018, pp. 2002–2011.

[33] A. Saxena, M. Sun, and A. Y. Ng, "Learning 3-d scene structure from a single still image," in *2007 IEEE 11th International Conference on Computer Vision*. IEEE, 2007, pp. 1–8.

[34] F. Tosi, F. Aleotti, M. Poggi, and S. Mattoccia, "Learning monocular depth estimation infusing traditional stereo knowledge," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2019.

[35] J. H. Lee, M.-K. Han, D. W. Ko, and I. H. Suh, "From big to small: Multi-scale local planar guidance for monocular depth estimation," *arXiv preprint arXiv:1907.10326*, 2019.

[36] C. Godard, O. Mac Aodha, M. Firman, and G. J. Brostow, "Digging into self-supervised monocular depth estimation," in *Proceedings of the IEEE international conference on computer vision*, 2019, pp. 3828–3838.

[37] J. Ens and P. Lawrence, "An investigation of methods for determining depth from focus," *IEEE Transactions on pattern analysis and machine intelligence*, vol. 15, no. 2, pp. 97–108, 1993.

[38] C.-H. Chen, H. Zhou, and T. Ahonen, "Blur-aware disparity estimation from defocus stereo images," in *Proceedings of the IEEE International Conference on Computer Vision*, 2015, pp. 855–863.

[39] S. Suwajanakorn, C. Hernandez, and S. M. Seitz, "Depth from focus with your mobile phone," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2015, pp. 3497–3506.

[40] G. Xu, Y. Quan, and H. Ji, "Estimating defocus blur via rank of local patches," in *Proceedings of the IEEE International Conference on Computer Vision*, 2017, pp. 5371–5379.

[41] L. Pan, Y. Dai, M. Liu, and F. Porikli, "Depth map completion by jointly exploiting blurry color images and sparse depth maps," in *2018 IEEE Winter Conference on Applications of Computer Vision (WACV)*. IEEE, 2018, pp. 1377–1386.

[42] M. Maximov, K. Galim, and L. Leal-Taixé, "Focus on defocus: bridging the synthetic to real domain gap for depth estimation," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2020, pp. 1071–1080.

[43] S. Zhuo and T. Sim, "Defocus map estimation from a single image," *Pattern Recognition*, vol. 44, no. 9, pp. 1852–1858, 2011.

[44] A. Levin, R. Fergus, F. Durand, and W. T. Freeman, "Image and depth from a conventional camera with a coded aperture," *ACM transactions on graphics (TOG)*, vol. 26, no. 3, pp. 70–es, 2007.

[45] J. Lee, S. Lee, S. Cho, and S. Lee, "Deep defocus map estimation using domain adaptation," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2019, pp. 12 222–12 230.

[46] Hyeongseok Son, J. Lee, S. Cho, and S. Lee, "Single image defocus deblurring using kernel-sharing parallel atrous convolutions," in *2021 IEEE/CVF International Conference on Computer Vision, ICCV 2021, Montreal, QC, Canada, October 10-17, 2021*. IEEE, 2021, pp. 2622–2630. [Online]. Available: https://doi.org/10.1109/ICCV48922.2021.00264

[47] J. Lee, H. Son, J. Rim, S. Cho, and S. Lee, "Iterative filter adaptive network for single image defocus deblurring," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2021, pp. 2034–2042.

[48] Y. Shirai, *Three-dimensional computer vision*. Springer Science & Business Media, 2012.

[49] R. I. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision – 2nd Edition*. Cambridge University Press, 2004.

[50] F. C. Crow, "Summed-area tables for texture mapping," in *Proceedings of the 11th annual conference on Computer graphics and interactive techniques*, 1984, pp. 207–212.

[51] S. Greene, "Summed area tables using graphics hardware," in *Game Developers Conference*, vol. 6, 2003, p. 8.

[52] P. Viola, M. Jones *et al.*, "Robust real-time object detection," *International journal of computer vision*, vol. 4, no. 34-47, p. 4, 2001.

[53] X. Cheng, Y. Zhong, M. Harandi, Y. Dai, X. Chang, T. Drummond, H. Li, and Z. Ge, "Hierarchical neural architecture search for deep stereo matching," *arXiv preprint arXiv:2010.13501*, 2020.

[54] H. Zhang, Y. Dai, H. Li, and P. Koniusz, "Deep stacked hierarchical multi-patch network for image deblurring," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2019.

[55] R. Girshick, "Fast r-cnn," in *Proceedings of the IEEE international conference on computer vision*, 2015, pp. 1440–1448.

[56] J. Fu, J. Liu, H. Tian, Y. Li, Y. Bao, Z. Fang, and H. Lu, "Dual attention network for scene segmentation," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2019, pp. 3146–3154.

[57] A. Karaali and C. R. Jung, "Edge-based defocus blur estimation with adaptive scale selection," *IEEE Transactions on Image Processing*, vol. 27, no. 3, pp. 1126–1137, 2017.

[58] V. Tankovich, C. Hane, Y. Zhang, A. Kowdle, S. Fanello, and S. Bouaziz, "Hitnet: Hierarchical iterative tile refinement network for real-time stereo matching," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2021, pp. 14 362–14 372.

[59] W. Yin, J. Zhang, O. Wang, S. Niklaus, L. Mai, S. Chen, and C. Shen, "Learning to recover 3d scene shape from a single image," in *Proc. IEEE Conf. Comp. Vis. Patt. Recogn. (CVPR)*, 2021.

[60] R. Zhang, P. Isola, A. A. Efros, E. Shechtman, and O. Wang, "The unreasonable effectiveness of deep features as a perceptual metric," in *CVPR*, 2018.

[61] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," *arXiv preprint arXiv:1409.1556*, 2014.

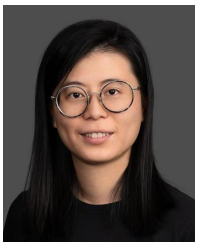
[62] O. Russakovsky, J. Deng, H. Su, J. Krause, S. Satheesh, S. Ma, Z. Huang, A. Karpathy, A. Khosla, M. Bernstein *et al.*, "Imagenet large scale visual recognition challenge," *International journal of computer vision*, vol. 115, no. 3, pp. 211–252, 2015.

[63] A. Krizhevsky, "One weird trick for parallelizing convolutional neural networks," *arXiv preprint arXiv:1404.5997*, 2014.

[64] P. K. Nathan Silberman, Derek Hoiem and R. Fergus, "Indoor segmentation and support inference from rgbd images," in *Proceedings of the European Conference on Computer Vision (ECCV)*, 2012.

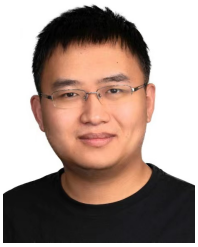
[65] D. Hernandez-Juarez, L. Schneider, A. Espinosa, D. Vázquez, A. M. López, U. Franke, M. Pollefeys, and J. C. Moure, "Slanted stixels: Representing san francisco's steepest streets," *arXiv preprint arXiv:1707.05397*, 2017.

[66] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *International Conference on Learning Representations (ICLR)*, 2015.

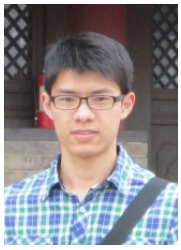


Liyuan Pan is currently an Associated Professor at the Beijing Institute of Technology, Beijing, China. She received her Ph.D. degree in the College of Engineering and Computer Science, Australian National University, Canberra, Australia in 2021. Her research interests include deblurring, flow estimation, depth estimation, and event-based vision.

Richard Hartley is a member of the computer vision group in the Research School of Engineering, ANU, where he has been since January, 2001. He is also a member of the computer vision research group in NICTA. He worked at the GE Research and Development Center from 1985 to 2001, working first in VLSI design, and later in computer vision. He became involved with Image Understanding and Scene Reconstruction working with GE's Simulation and Control Systems Division. He is an author (with A. Zisserman) of the book Multiple View Geometry in Computer Vision. He is a Fellow of the IEEE.



Liu Liu is a researcher with Huawei, China. He received his Ph.D. degree in the College of Engineering and Computer Science, Australian National University, Canberra, Australia in 2021. His research interests include 3D geometry (e.g., reconstruction and localization) and scene understanding problems.



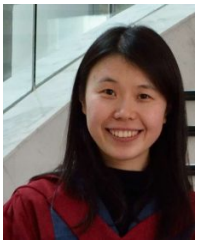
Zhiwei Xu received his B.E. degree from Huaqiao University, Xiamen, China in 2012, S.E. degree from Xidian University, Xi'an, China in 2015, and his Ph.D. degree in the College of Engineering and Computer Science, Australian National University, Canberra, Australia in 2021. His interests include Markov random field optimization, semantic segmentation, and stereo vision.

Shah Chowdhury received his B.E. degree and M.E. degree from Rajshahi University of Engineering and Technology (RUET), Bangladesh in 2011 and 2014, and his Ph.D. degree in the College of Engineering and Computer Science, Australian National University, Canberra, Australia in 2021. His interests include dual-pixel sensor and 3D reconstruction.

Yan Yang is currently pursuing the Ph.D. degree in the Australian National University, Canberra, Australia. He received her B.E degree from the Australian National University, Canberra, Australia in 2020. His interests interests include event cameras and pathology images.

Hongguang Zhang is an Assistant Professor in computer vision and machine learning at the Systems Engineering Institute, AMS. He received the BSc degree in electrical engineering and automation from Shanghai Jiao Tong University, Shanghai, China, in 2014, received his MSc degree in electronic science and technology from National University of Defense Technology, Changsha, China, in 2016, and received his PhD degree from the Australian National University and Data61/CSIRO, Canberra, Australia in 2020. His interests include fine-grained image classification, zero-shot learning, few-shot learning, low-level vision and deep learning.

Hongdong Li is a Professor with the Computer Vision Group, the Australian National University. He is also a Chief Investigator of the Australia ARC Centre of Excellence for Robotic Vision. Formerly, he was a Distinguished Scientist with Tencent, as the Research Director for Tencent's XR (VR/AR) program. His research interests include 3D computer vision, visual perception, 3D modeling, robot navigation and autonomous driving, virtual and augmented reality. He has been an Associate Editor sitting on the Editorial board for IEEE T-PAMI. He has won multiple prestigious paper awards in computer vision, including the IEEE CVPR Best Paper Award and the Marr Prize (Honorable Mention) etc. He is currently working on generative AI projects for 3D content generation and visual scene understanding.



Miaomiao Liu received the BEng, MEng, and PhD degrees from Yantai Normal University, Yantai, China, Nanjing University of Aeronautics and Astronautics, Nanjing, China, and the University of Hong Kong, Hong Kong, China in 2004, 2007, and 2012, respectively. She worked as a researcher in the computer vision group at NICTA (2012-2016) and a research scientist (2017-2018) at CSIRO's Data61 in Canberra, Australia. She joined the Australian National University (ANU) as an ARC DECRA fellow in 2018. She is currently a Senior Lecturer in the School of Computing, ANU. She is a Member of the IEEE.